

Optimization-Based Autonomous Racing of 1:43 Scale RC Cars

Alexander Liniger, Alexander Domahidi and Manfred Morari

Abstract

This paper describes autonomous racing of RC race cars based on mathematical optimization. Using a dynamical model of the vehicle, control inputs are computed by receding horizon based controllers, where the objective is to maximize progress on the track subject to the requirement of staying on the track and avoiding opponents. Two different control formulations are presented. The first controller employs a two-level structure, consisting of a path planner and a nonlinear model predictive controller (NMPC) for tracking. The second controller combines both tasks in one nonlinear optimization problem (NLP) following the ideas of contouring control. Linear time varying models obtained by linearization are used to build local approximations of the control NLPs in the form of convex quadratic programs (QPs) at each sampling time. The resulting QPs have a typical MPC structure and can be solved in the range of milliseconds by recent structure exploiting solvers, which is key to the real-time feasibility of the overall control scheme. Obstacle avoidance is incorporated by means of a high-level corridor planner based on dynamic programming, which generates convex constraints for the controllers according to the current position of opponents and the track layout. The control performance is investigated experimentally using 1:43 scale RC race cars, driven at speeds of more than 3 m/s and in operating regions with saturated rear tire forces (drifting). The algorithms run at 50 Hz sampling rate on embedded computing platforms, demonstrating the real-time feasibility and high performance of optimization-based approaches for autonomous racing.

I. INTRODUCTION

Autonomous car racing is a challenging task for automatic control systems due to the need for handling the vehicle close to its stability limits and in highly nonlinear operating regimes. In addition, dynamically changing racing situations require advanced path planning mechanisms with obstacle avoidance executed in real-time. Fast dynamics constrain the sampling time to be in the range of a few tens of milliseconds at most, which severely limits the admissible computational complexity of the algorithms. This situation is even more challenging if the autonomous algorithms shall be executed on simple, low-power embedded computing platforms.

In this paper, we investigate optimization-based control strategies for the task of racing an autonomous vehicle around a given track. We focus on methods that can be implemented to run in real-time on embedded control platforms, and present experimental results using 1:43 scale Kyosho *dnano* RC race cars that achieve top speeds of more than 3 m/s, which corresponds to an upscaled speed of about 465 km/h. For high performance, the proposed controllers operate the car at its friction limits, far beyond the linear region of what is typically used in other autonomous driving systems. This challenging task is generally mastered only by expert drivers with lots of training. In contrast, our approach requires merely a map of the track and a dynamical model of the car; in particular, we use a bicycle model with nonlinear tire forces, neglecting load transfer and coupling of lateral and longitudinal slip.

Two model-based control schemes are presented in this paper. First, we describe a hierarchical two-level control scheme consisting of a model-based path planner generating feasible trajectories for an underlying nonlinear model predictive control (NMPC) trajectory tracking controller. Our second approach combines both path planning and path following into one formulation, resulting in one NMPC controller that is based on a particular formulation known from contouring control [1], [2]. The objective of both approaches is to maximize the progress on the track, measured by a projection of the vehicle's position onto the center line of the track. Linear time varying (LTV) models obtained by linearization are employed to construct a

基于优化的1:43比例遥控车自主赛车

亚历山大·林格尔、亚历山大·多马希迪 和 曼弗雷德·莫拉里

摘要

本文描述了基于数学优化的RC赛车自主竞速。利用车辆的动力学模型，通过递推视野控制器计算控制输入，其目标是在保持在赛道上并避开对手的前提下最大化赛道上的进度。文中提出了两种不同的控制方法。第一种控制器采用两级结构，由路径规划器和用于跟踪的非线性模型预测控制器（NMPC）组成。第二种控制器将两项任务结合在一个非线性优化问题（NLP）中，遵循轮廓控制的理念。通过线性化获得的线性时变模型用于在每个采样时间建立控制NLP的局部近似，形式为凸二次规划（QPs）。生成的QPs具有典型的MPC结构，并且可以通过最近的结构利用求解器在毫秒级内求解，这对于实时性是关键。

一、引言

自动驾驶赛车对自动控制系统来说是一项具有挑战性的任务，因为需要在接近车辆稳定性极限和高度非线性操作状态下控制车辆。此外，动态变化的赛车情况需要先进的路径规划机制，并能够实时执行避障操作。快速的动态特性限制了采样时间最多只能在几十毫秒的范围内，这严重限制了算法可允许的计算复杂性。如果自主算法还需要在简单、低功耗的嵌入式计算平台上运行，这种情况将更加具有挑战性。

在本文中，我们研究了用于自动驾驶车辆在给定赛道上赛车任务的基于优化的控制策略。我们关注那些可以在嵌入式控制平台上实时运行的方法，并展示了使用1:43比例Kyosho *dnano* RC赛车的实验结果，这些赛车的最高速度超过3米/秒，相当于放大后的速度约为465公里/小时。为了实现高性能，所提出的控制器将车辆操作在其摩擦极限范围内，远超通常在其他自动驾驶系统中使用的线性区间。这个具有挑战性的任务通常只有经过大量训练的专业驾驶员才能掌握。相比之下，我们的方法仅需赛道地图和车辆动力学模型；特别地，我们使用带有非线性轮胎力的自行车模型，并忽略载荷转移以及横向与纵向滑移的耦合。

本文提出了两种基于模型的控制方案。首先，我们描述了一种分层的两级控制方案，包括一个基于模型的路径规划器，为底层的非线性模型预测控制（NMPC）轨迹跟踪控制器生成可行轨迹。我们的第二种方法将路径规划和路径跟随结合到一个公式中，从而得到一个基于路径跟踪控制已知特定公式的NMPC控制器 [1], [2]。这两种方法的目标都是最大化车辆在赛道上的进度，以车辆位置在赛道中心线上的投影来衡量。通过线性化得到的线性时变（LTV）模型被用来构建一个

tractable convex optimization problem to be solved at each sampling time. Efficient interior point solvers for embedded systems, generated by FORCES [3], [4], are employed to solve the resulting optimization problems, which makes the approaches presented in this paper amenable for use on embedded systems.

We furthermore demonstrate that both schemes can easily be extended to incorporate obstacle avoidance by adjusting the constraints of the resulting optimization problems according to the racing situation. With this mechanism in place, the avoidance trajectory is optimized to yield maximal overall progress, which translates into highly effective overtaking maneuvers that are automatically planned and executed.

A. Related work

Safe autonomous driving at moderate speeds has been demonstrated for example in the context of Autonomous Highway Systems (AHS) in 1997 [5] within the Californian PATH programme, or in contests such as the DARPA Grand Challenge in 2005 [6] and the Urban Challenge in 2007 [7] by numerous research groups. In these projects, the main goal was to develop autonomous driving systems for public traffic situations. Autonomous racing has received less attention, but impressive advances have been made recently also in this discipline. For instance, a fully autonomous Audi TTS has been reported to race at high speeds in [8] using a trajectory tracking controller, which follows a pre-computed trajectory. However, to the best knowledge of the authors, fully automatic racing including obstacle avoidance in dynamic racing situations has not been demonstrated yet in practice.

From a control perspective, the fundamental building blocks for automatic racing can be grouped into three categories: drift control, reference tracking, and path planning. Research associated with the first group focuses on gaining a better understanding of the behavior of the car near the friction limit for designing safety control systems that try to prevent loss-of-control accidents. By analyzing the nonlinear car model, it can be shown that steady state motions corresponding to drifting can be generated, see e.g. [9] and [10]. As these approaches are not designed for trajectory tracking, the controlled car drifts in a circle, which represents a steady state drift equilibrium.

Reference tracking controllers usually are designed to operate the vehicle within the linear tire force region for maximum safety and robustness. Approaches based on NMPC, which allows one to incorporate the latter constraint in a systematic manner, deal with the nonlinearities of the model either directly by a nonlinear programming (NLP) solver [11], or use LTV [12] or piece-wise affine (PWA) approximations [13], resulting in a convex quadratic program (QP) or a mixed-integer QP, respectively. Approaches without optimization in real-time include nonlinear robust control [14] and flatness-based control [15], which however lack the ability to incorporate constraints. One approach that is explicitly designed for operating with saturated tire forces is [8], where the center of percussion (CoP) is used to design a state feedback steering controller for reference tracking. At the CoP, the rear tire forces do not influence the dynamics by definition, which allows for a simple linear steering controller.

In order to avoid obstacles, reference tracking schemes rely on a higher-level path planner. A simple point mass model in the high-level path planner is used in [16] to limit the computational complexity. This can be problematic if the generated trajectories are infeasible with respect to the car's physical dynamics, which can lead to preventable collisions [17]. The latter reference suggests to use a library of steady-state movements (trims) with parametrizable length, which are generated from a more complex dynamical model, and to connect them by maneuvers which allow the transition from one trim to another, similar to the idea of [18]. This path planning has the advantage that it generates feasible reference trajectories for the low level tracking controller, and that drifting trims can be included to increase the agility of the car. However, the resulting optimization problem is a mixed-integer program that is too complex to be solved within the typical sampling times on embedded platforms. Consequently, the approach is not well suited for real-time autonomous racing.

In order to avoid relying on the feasibility of the path planner, one-level approaches have been investigated e.g. in [19], where optimal trajectories and the associated open-loop control inputs for rally racing maneuvers are calculated numerically in simulation. In [16], obstacle avoidance is incorporated into

在每个采样时间需要解决的可处理的凸优化问题。使用由 FORCES [3], [4] 生成的适用于嵌入式系统的高效内点求解器来解决由此产生的优化问题, 这使得本文提出的方法适合在嵌入式系统上使用。

此外, 我们证明了这两种方案都可以通过根据赛道情况调整由此产生的优化问题的约束, 轻松扩展以纳入障碍物避让。在该机制到位的情况下, 避让轨迹被优化以实现最大整体进展, 这转化为高度有效的超车操作, 这些操作可以自动规划和执行。

A. Related work

例如, 在1997年加利福尼亚PATH项目中的自动高速公路系统(AHS)背景下, 已经展示了在中等速度下的安全自动驾驶[5], 或者在2005年的DARPA大挑战赛[6]和2007年的城市挑战赛[7]等比赛中, 由众多研究团队进行了展示。在这些项目中, 主要目标是开发用于公共交通情境的自动驾驶系统。自动驾驶赛车受到的关注较少, 但在这一领域最近也取得了令人印象深刻的进展。例如, 据报道, 一辆完全自主的奥迪TTS能够在[8]中以高速进行赛车比赛, 使用的是轨迹跟踪控制器, 该控制器遵循预先计算的轨迹。然而, 据作者所知, 完全自动的赛车, 包括在动态赛况下的障碍物规避, 目前在实际中尚未得到验证。

从控制的角度来看, 自动赛车的基本构建模块可以分为三类: 漂移控制、参考跟踪和路径规划。与第一类相关的研究侧重于更好地理解汽车在摩擦极限附近的行为, 以设计旨在防止失控事故的安全控制系统。通过分析非线性汽车模型, 可以表明可以生成对应于漂移的稳态运动, 例如见[9]和[10]。由于这些方法并非为轨迹跟踪设计, 受控汽车会在一个圆圈中漂移, 这代表了一个稳态漂移平衡。

参考跟踪控制器通常被设计用于在轮胎线性力区域内操作车辆, 以获得最大的安全性和鲁棒性。基于非线性模型预测控制(NMPC)的方法, 可以系统地将后者约束纳入, 处理模型的非线性要么直接通过非线性规划(NLP)求解器[11], 要么使用时变线性(LTV)[12]或分段仿射(PWA)逼近[13], 分别得到凸二次规划(QP)或混合整数二次规划。实时不进行优化的方法包括非线性鲁棒控制[14]和基于平坦性的控制[15], 然而这些方法缺乏结合约束的能力。一种专门用于在饱和轮胎力条件下操作的方法是[8], 其中使用击打中心(CoP)设计用于参考跟踪的状态反馈转向控制器。在CoP, 后轮力按定义不会影响动力学, 从而允许设计简单的线性转向控制器。

为了避免障碍物, 参考跟踪方案依赖于高层路径规划器。在[16]中, 高层路径规划器中使用了一个简单的质点模型来限制计算复杂性。如果生成的轨迹在汽车的物理动力学上不可行, 这可能会成为一个问题, 从而导致可以避免的碰撞[17]。后一篇文献建议使用一个稳态运动(修剪)的库, 这些运动具有可参数化的长度, 由更复杂的动力学模型生成, 并通过允许从一个修剪过渡到另一个修剪的机动来连接它们, 这与[18]的想法类似。这种路径规划的优点是它为低层跟踪控制器生成了可行的参考轨迹, 并且可以包含漂移修剪以提高汽车的灵活性。然而, 所得到的优化问题是一个混合整数规划, 其复杂度过高, 无法在嵌入式系统的典型采样时间内解决。

为了避免依赖路径规划器的可行性, 人们已经研究了一层方法, 例如在[19]中, 拉力赛车动作的最优轨迹及相关的开环控制输入在仿真中通过数值计算得到。在[16]中, 障碍物规避被纳入

the tracking controller by using an additional cost term that penalizes coming too close to the obstacle. However, the controller is reported to perform worse than its two-level counterpart, especially if the car has to diverge from the reference trajectory. A one-level approach similar to [16] is studied in [20] in simulation, where obstacle avoidance is incorporated into the problem formulation by using a spatial reformulation and imposing constraints on the transformed variables. While in [19], [16] the solution to the NLP is obtained by a standard nonlinear solver with prohibitively long solve times, [20] uses real-time iterations [21] to achieve the low computation times needed for a real-time implementation. However, it is assumed that there is only one side where the obstacle can be overtaken, which may not be the case in practical racing situations.

An interesting alternative to optimization-based methods are sampling-based methods. For example, rapidly exploring random trees (RRTs) have been investigated in [22] and [23] to generate time optimal trajectories through a 180° curve. The advantage is that such algorithms tend to quickly find feasible trajectories. In [23], the differential flatness of the system is exploited to speed up the convergence of the algorithm, generating obstacle-free trajectories even in complex situations. However, the reported computation times are not yet low enough to allow for a real-time implementation.

B. Contribution

In this paper, we describe two novel autonomous, optimization-based racing controllers that incorporate obstacle avoidance, track constraints, actuator limitation and the ability for controlled drift in a systematic and straightforward way. Real-time feasibility at 50Hz sampling rate is demonstrated in experimental results with fast RC cars. To the best knowledge of the authors, this is the first implementation of autonomous racing controllers with obstacle avoidance on an experimental testbed.

The fundamental idea of both approaches is to use a receding horizon controller, which maximizes progress on the track within the horizon as a performance measure. This is closely related to a time optimality criterion and allows for a systematic incorporation of obstacles and other constraints. It is particularly effective for overtaking opponents, as our controllers seek for a progress-optimal solution around the obstacles. In order to deal with track constraints and obstacle avoidance situations, we represent the feasible set for the position of the car at any time instance by a slab defined by two parallel linear inequalities. This ensures tractability of the subproblems by convex programming on one hand, and incorporation of track and obstacle constraints in a systematic manner on the other hand. To deal with non-convex situations such as overtaking an obstacle on the left or right, a high-level corridor planning algorithm supplies a set of appropriate convex constraints to the controllers. This constraint set is the result of a shortest path problem solved by dynamic programming.

The first, hierarchical control approach presented in this paper is in principle similar to [17], but its complexity is low enough to be fully implemented in real-time. Our path planner also uses trims, but only *one* is selected for the whole prediction horizon from a set of trajectories, which represent steady-state cornering conditions of the nonlinear model, gridded for velocity and steering angle. The path planner selects the trajectory with the largest progress that does not leave the track or hits any obstacle. Moving obstacles can inherently be dealt with, as new trims are generated every sampling time. A low level nonlinear reference tracking MPC controller tracks the selected trim, as in [17], but we additionally use obstacle avoidance constraints as described above, which turns out to be very effective in practice.

The second controller is based on a model predictive contouring control (MPCC) formulation [1], [2], combining path generation and path tracking into one problem. The MPCC essentially plans a progress-optimal path by taking into account the (nonlinear) projection of the vehicle's position onto the center line. The resulting controller is able to plan and to follow a path which is similar to the time-optimal path in [19] when the horizon is chosen long enough.

Both NMPC problems are approximated locally by linearizing the continuous nonlinear system dynamics around a state trajectory to obtain an LTV model. In the hierarchical approach, we linearize around the trajectory obtained from the path planner, while in the one-level approach the linearization points are given

通过使用一个附加的代价项来惩罚接近障碍物，可以对跟踪控制器进行改进。然而，据报道该控制器的表现不如其两级对应物，尤其是在汽车必须偏离参考轨迹时。[20]中研究了一种类似于[16]的单级方法的仿真，在该方法中，通过空间重构并对变换后的变量施加约束，将避障纳入问题的公式化中。而在[19]、[16]中，非线性规划（NLP）的解是通过标准非线性求解器获得的，这会导致求解时间过长，[20]则使用实时迭代[21]来实现实时应用所需的低计算时间。然而，假设障碍物只存在于可以超车的一侧，而在实际赛车情况下可能并非如此。

优化方法的一个有趣替代方案是基于采样的方法。例如，快速扩展随机树（RRT）已在[22]和[23]中被研究，用于生成通过180°曲线的时间最优轨迹。其优点是此类算法通常能够快速找到可行轨迹。在[23]中，利用系统的微分平坦性来加速算法收敛，即使在复杂情况下也能生成无障碍轨迹。然而，报道的计算时间仍然不足以实现实时应用。

B. Contribution

在本文中，我们描述了两款新型的基于优化的自主赛车控制器，这些控制器以系统且直接的方式结合了障碍物规避、赛道约束、执行器限制以及受控漂移能力。实验结果表明，在快速遥控车上以50 Hz的采样率实现了实时可行性。据作者所知，这是首次在实验测试平台上实现具有障碍物规避功能的自主赛车控制器。

两种方法的基本思想都是使用递退地平线控制器，该控制器在该地平线范围内以进度最大化作为性能指标。这与时间最优准则密切相关，并允许系统地引入障碍物和其他约束条件。它在超越对手时尤其有效，因为我们的控制器会寻找绕过障碍物的进度最优解决方案。为了处理赛道约束和障碍物规避情况，我们用由两条平行线性不等式定义的条带来表示汽车在任意时刻的位置可行集。这一方法一方面通过凸规划确保子问题的可解性，另一方面以系统的方式纳入赛道和障碍物约束。为了处理如从左侧或右侧超越障碍物等非凸情况，高级通道规划算法提供了一组适当的凸约束给控制器。

本文提出的第一个分层控制方法在原理上与[17]类似，但其复杂性足够低，可实现实时完整应用。我们的路径规划器也使用修整（trims），但在整个预测时间范围内只从轨迹集合中选择one，这些轨迹表示非线性模型的稳态转弯条件，并针对速度和转向角进行了网格化。路径规划器会选择在不离开赛道或碰到任何障碍的情况下进展最大的轨迹。移动障碍物可以被本质上处理，因为每次采样时间都会生成新的修整。一个低级非线性参考跟踪MPC控制器跟踪所选修整，如[17]中所述，但我们额外使用了上述描述的障碍规避约束，事实证明在实践中非常有效。

第二个控制器基于模型预测轮廓控制（MPCC）形式[1]，[2]，将路径生成和路径跟踪合并为一个问题。MPCC本质上通过考虑车辆位置在中心线上的（非线性）投影来规划进度最优路径。生成的控制器能够规划并跟随一条路径，当预测期足够长时，该路径与[19]中的时间最优路径相似。

两个NMPC问题都通过在状态轨迹周围线性化连续非线性系统动力学来在局部进行近似，以获得一个LTV模型。在分层方法中，我们围绕从路径规划器获得的轨迹进行线性化，而在单层方法中，线性化点是预先给定的。

by the shifted trajectory from the previous sampling time. We then discretize by the matrix exponential, and solve the resulting quadratic program (QP) by a tailored interior point solver, generated by FORCES [3], [4]. The computation time of the solvers scale linearly with the horizon length, hence we make use of long horizons of up to 40 time steps in the MPCC, for example. After the QP has been solved, the first control input is applied to the system, and the process is repeated at the next sampling time. This approach for solving the NMPC problems corresponds to the basic version of the real-time iterations from [21].

C. Outline

The paper is organized as follows. In Section II, the dynamical model of the car and some of its basic properties are described. In Section III-A, we present the hierarchical control approach with separate path planner and path tracking, while the combined approach is presented in Section III-B. In Section III-C, we briefly discuss our method for selecting convex constraints for the two controllers, enabling obstacle avoidance in dynamic racing situations. Section IV presents the testbed setup and experimental results for both controllers.

NOTATION

The following notation is used throughout the paper. The set of reals is denoted by $\mathbb{R}$, and the set of real column vectors of length n is denoted by $\mathbb{R}^n$. The set of nonnegative and strictly positive reals is denoted by $\mathbb{R}_+$ and $\mathbb{R}_{++}$, respectively. The set of positive definite matrices of size n is denoted by $\mathbb{S}_{++}^n$ and the set of positive semi-definite matrices of size n by $\mathbb{S}_+^n$. We define $\|x\|_P^2 \triangleq x^T P x$ for a vector $x \in \mathbb{R}^n$ and some positive definite matrix $P \in \mathbb{S}_{++}^n$. If $a \in \mathbb{R}^n$ and $b \in \mathbb{R}^m$ are column vectors or scalars, we denote their concatenation by $(a, b) \triangleq [a^T, b^T]^T \in \mathbb{R}^{n+m}$.

II. CAR MODEL

A. Bicycle Model

The RC cars are modeled using a bicycle model as done in [9], [10], where the car is modeled as one rigid body with a mass m and an inertia I_z , and the symmetry of the car is used to reduce it to a bicycle. Only the in-plane motions are considered, i.e. the pitch and roll dynamics as well as load changes are neglected. As the used cars are rear wheel driven and do not have active brakes, the longitudinal force on the front wheel is neglected. The resulting model is shown in Figure 1 together with the resulting differential equations (1) defining the model:

$$\dot{X} = v_x \cos(\varphi) - v_y \sin(\varphi), \quad (1a)$$

$$\dot{Y} = v_x \sin(\varphi) + v_y \cos(\varphi), \quad (1b)$$

$$\dot{\varphi} = \omega, \quad (1c)$$

$$\dot{v}_x = \frac{1}{m}(F_{r,x} - F_{f,y} \sin \delta + m v_y \omega), \quad (1d)$$

$$\dot{v}_y = \frac{1}{m}(F_{r,y} + F_{f,y} \cos \delta - m v_x \omega), \quad (1e)$$

$$\dot{\omega} = \frac{1}{I_z}(F_{f,y} l_f \cos \delta - F_{r,y} l_r). \quad (1f)$$

The equation of motion is derived around the center of gravity (CoG), where the states are the position X , Y of the CoG in the inertial frame and the angle of the car relative to the inertial frame, φ . This is the kinematic part of the model. The kinetic part of the model is derived around a body-fixed frame centered at the CoG, where the states are the longitudinal and lateral velocity of the car, v_x and v_y , and finally the yaw rate ω . The control inputs are the PWM duty cycle d of the electric drive train motor and

由上一次采样时间的偏移轨迹得到。然后通过矩阵指数进行离散化，并通过由 FORCES [3], [4] 生成的定制内点求解器求解得到的二次规划（QP）。求解器的计算时间与预测时域长度线性相关，因此我们在 MPCC 中使用了长达 40 个时间步的长时域。例如，在 QP 被求解后，第一个控制输入被施加到系统上，并在下一次采样时间重复该过程。该方法用于求解 NMPC 问题，对应于文献 [21] 中实时迭代的基本版本。

C. Outline

本文的组织结构如下：在第二部分中，描述了汽车的动力学模型及其一些基本特性。在第三部分 A 节中，我们提出了具有独立路径规划器和路径跟踪器的分层控制方法，而结合方法则在第三部分 B 节中介绍。在第三部分 C 节中，我们简要讨论了选择两个控制器的凸约束的方法，使其能够在动态赛车情况下实现避障。第四部分介绍了测试平台的设置以及两个控制器的实验结果。

符号

本文中使用的符号。实数集合记作 $\mathbb{R}$ ，长度为 n 的实列向量集合记作 $\mathbb{R}^n$ 。非负实数集合记作 $\mathbb{R}_+$ ，严格正实数集合记作 $\mathbb{R}_{++}$ 。大小为 n 的正定矩阵集合记作 $\mathbb{S}_{++}^n$ ，大小为 n 的半正定矩阵集合记作 $\mathbb{S}_+^n$ 。对于向量 $x \in \mathbb{R}^n$ 和某个正定矩阵 $P \in \mathbb{S}_{++}^n$ ，我们定义 $\|x\|_P^2 \triangleq x^T P x$ 。如果 $a \in \mathbb{R}^n$ 和 $b \in \mathbb{R}^m$ 是列向量或标量，我们用 $(a, b) \triangleq [a^T, b^T]^T \in \mathbb{R}^{n+m}$ 表示它们的连接。

II. CAR MODEL

A. Bicycle Model

RC 车使用自行车模型进行建模，如文献 [9]、[10] 所述，其中汽车被建模为具有质量 m 和转动惯量 I_z 的一个刚体，并利用汽车的对称性将其简化为自行车模型。仅考虑平面内的运动，即忽略俯仰和横滚动力学以及载荷变化。由于所使用的车为后轮驱动且没有主动制动，前轮的纵向力被忽略。所得模型如图 1 所示，同时给出了定义该模型的微分方程 (1):

$$\dot{X} = v_x \cos(\varphi) - v_y \sin(\varphi), \quad (1a)$$

$$\dot{Y} = v_x \sin(\varphi) + v_y \cos(\varphi), \quad (1b)$$

$$\dot{\varphi} = \omega, \quad (1c)$$

$$\dot{v}_x = \frac{1}{m}(F_{r,x} - F_{f,y} \sin \delta + m v_y \omega), \quad (1d)$$

$$\dot{v}_y = \frac{1}{m}(F_{r,y} + F_{f,y} \cos \delta - m v_x \omega), \quad (1e)$$

$$\dot{\omega} = \frac{1}{I_z}(F_{f,y} l_f \cos \delta - F_{r,y} l_r). \quad (1f)$$

运动方程是围绕重心（CoG）导出的，其中状态是重心在惯性参考系中的位置 X 、 Y 以及汽车相对于惯性参考系的角度 φ 。这是模型的运动学部分。模型的动力学部分是围绕以重心为中心的车体固定参考系导出的，其中状态是汽车的纵向和横向速度 v_x 和 v_y ，最后是偏航率 ω 。控制输入是电驱动系统电机的 PWM 占空比 d ，以及

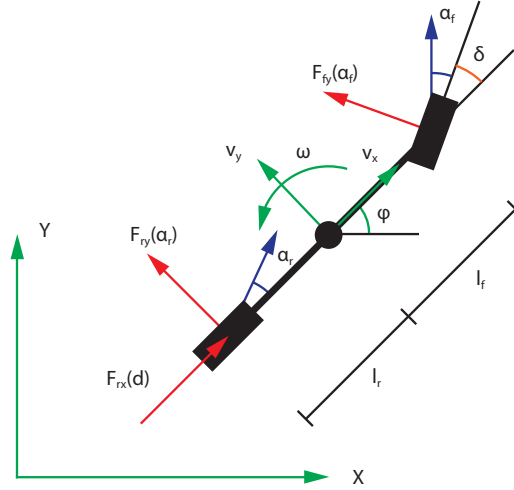


Fig. 1: Schematic drawing of the car model

the steering angle δ . The subscripts x and y indicate longitudinal and lateral forces or velocities, while r and f refer to the front and rear tires, respectively. Finally, l_f and l_r are the distance from the CoG to the front and the rear wheel.

The tire forces F model the interaction between the car and the road and are the most important part of the dynamics. As the goal is for the cars to race, the model of the tire forces has to be realistic enough to represent the car at high speeds and its handling limits. The lateral forces $F_{f,y}$ and $F_{r,y}$ are modeled using a simplified Pacejka Tire Model [24], see (2a) and (2b), which is a good approximation to measured friction curves in practice:

$$F_{f,y} = D_f \sin(C_f \arctan(B_f \alpha_f)) \quad \text{where} \quad \alpha_f = -\arctan\left(\frac{\omega l_f + v_y}{v_x}\right) + \delta, \quad (2a)$$

$$F_{r,y} = D_r \sin(C_r \arctan(B_r \alpha_r)) \quad \text{where} \quad \alpha_r = \arctan\left(\frac{\omega l_r - v_y}{v_x}\right), \quad (2b)$$

$$F_{r,x} = (C_{m1} - C_{m2}v_x)d - C_r - C_d v_x^2. \quad (2c)$$

The parameters B , C and D define the exact shape of the semi-empirical curve. The longitudinal force of the rear wheel $F_{r,x}$ is modeled using a motor model for the DC electric motor as well as a friction model for the rolling resistance and the drag, cf. (2c). Due to the size of the cars, it is currently not feasible to measure the wheel speed, which makes it impossible to use combined slip models such as [9], [19].

The model is identified using a combination of stationary and dynamic identification experiments, as reported in [10]. This allows for identifying the rear wheel combined slip effects into the lateral tire friction model (2b). Thus the identified model is suitable to represent the car also at its handling limits, when the tire forces are saturated or close to saturation.

B. Stationary Velocity Analysis

For a better understanding of the model, the stationary velocities of the model are investigated. A similar, more elaborate analysis is presented in [9], [10].

The objective is to find points in the model where all accelerations are zero. This is done for different constant forward velocities $\bar{v}_x$ and constant steering angles $\bar{\delta}$, thus the problem becomes a nonlinear algebraic system of equations, with two equations ($\dot{v}_y = 0$, $\dot{\omega} = 0$) and two unknowns ($\bar{v}_y$, $\bar{\omega}$). By finding a solution to the algebraic equations for different steering angles $\bar{\delta}$ and different initial conditions, the stationary velocities for one forward velocity ($\bar{v}_y$, $\bar{\omega}$) can be calculated.

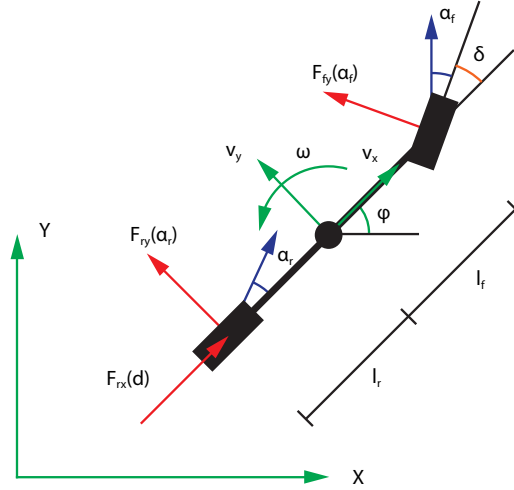


图 1: 汽车模型的示意图

转向角 δ 。下标 x 和 y 表示纵向和横向的力或速度，而 r 和 f 分别指前轮和后轮。最后， l_f 和 l_r 是从质心到前轮和后轮的距离。

轮胎力 F 模拟汽车与道路之间的相互作用，是动力学中最重要的部分。由于目标是让汽车进行比赛，轮胎力的模型必须足够逼真，以能够表现汽车在高速下的性能及其操控极限。侧向力 $F_{f,y}$ 和 $F_{r,y}$ 使用简化的Pacejka轮胎模型 [24] 进行建模，见 (2a) 和 (2b)，在实际中这是对实测摩擦曲线的良好近似：

$$F_{f,y} = D_f \sin(C_f \arctan(B_f \alpha_f)) \quad \text{where} \quad \alpha_f = -\arctan\left(\frac{\omega l_f + v_y}{v_x}\right) + \delta, \quad (2a)$$

$$F_{r,y} = D_r \sin(C_r \arctan(B_r \alpha_r)) \quad \text{where} \quad \alpha_r = \arctan\left(\frac{\omega l_r - v_y}{v_x}\right), \quad (2b)$$

$$F_{r,x} = (C_{m1} - C_{m2}v_x)d - C_r - C_d v_x^2. \quad (2c)$$

参数 B 、 C 和 D 定义了半经验曲线的精确形状。后轮的纵向力 $F_{r,x}$ 是通过直流电动机的电机模型以及滚动阻力和阻力的摩擦模型进行建模的，参见 (2c)。由于车辆的尺寸，目前无法测量车轮速度，这使得无法使用诸如 [9]、[19] 的联合滑移模型。

该模型是通过结合静态和动态识别实验进行识别的，如文献[10]所报道。这允许将后轮的组合滑移效应识别到横向轮胎摩擦模型 (2b) 中。因此，该识别的模型适合在车辆操控极限时表示汽车的行为，即当轮胎力达到饱和或接近饱和时。

B. Stationary Velocity Analysis

为了更好地理解该模型，对模型的稳态速度进行了研究。在文献 [9]、[10] 中给出了类似的、更详细的分析。

目标是找到模型中所有加速度为零的点。这是针对不同的恒定前进速度 $\bar{v}_x$ 和恒定转向角 $\bar{\delta}$ 进行的，因此问题变成了一个非线性代数方程组，有两个方程 ($\dot{v}_y = 0, \dot{\omega} = 0$) 和两个未知数 ($\bar{v}_y, \bar{\omega}$)。通过为不同的转向角 $\bar{\delta}$ 和不同的初始条件求解代数方程，可以计算出一个前进速度 ($\bar{v}_y, \bar{\omega}$) 下的静止速度。

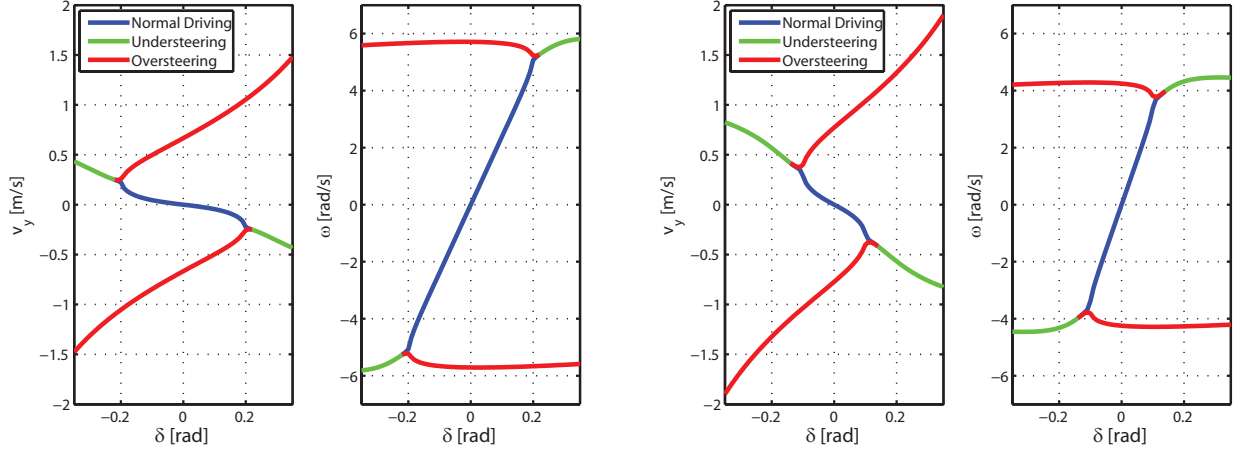


Fig. 2: Stationary velocities for $\bar{v}_x = 1.5$ m/s (plots on the left) and $\bar{v}_x = 2$ m/s (plots on the right), parameterized by the steering angle $\bar{\delta}$.

The resulting stationary velocities are shown for $\bar{v}_x = 1.5$ m/s and $\bar{v}_x = 2$ m/s in Figure 2. The resulting stationary velocities can be categorized into three different regions. The blue line is the normal driving region, which is characterized by the linear dependency of the yaw rate and the steering angle, as well as small lateral velocities. The other two regions in red and green correspond to over- and understeering points in the model. In the understeering region (green curves), the saturated front tire force law prevents the car from achieving larger yaw rates, and thus the car can not take a sharper turn. In the oversteering region (red curves), the rear tire force law is fully saturated and the car drives with a high lateral velocity, which is usually called drift.

The dynamics of the model in the different regions are different, for example in the oversteering region where the car is drifting, the steering angle can be of opposite sign than the curvature the car is driving, known as counter steering, which does not occur in the case of the normal and the understeering mode. A stability analysis of the lateral velocity model around the different stationary velocity points also shows the difference in the dynamics: While the normal driving region and the understeering region are stable, the drifting region is unstable [10].

From Figure 2 it is visible that the normal steering region is getting smaller when the car is driving faster. Thus the maximal and minimal steering angle for which the tire forces are not saturated is reduced with increasing velocities, and at the same time the maximal yaw rate decreases.

Because all velocities are constant, the movement of the car is a uniform circular movement, for which the relationship between the yaw rate, the radius R and the curvature κ is known,

$$|\omega| = \frac{v}{R} = v\kappa \quad \text{where} \quad v = \sqrt{v_x^2 + v_y^2}. \quad (3)$$

Consequently, all stationary velocity points correspond to the car driving in a circle with a constant radius, i.e. the model can predict drifting along a circle.

III. AUTONOMOUS RACING CONTROL

In this main part of the paper, we present two optimization-based formulations for the task of autonomous racing for RC cars which are based on the model described in Section II. First, the hierarchical two-level approach is presented in Section III-A, followed by the one-level scheme in Section III-B. We then briefly discuss a high-level method for obstacle avoidance that is the same for both controllers in Section III-C. A summary of the algorithms is given in Section III-D.

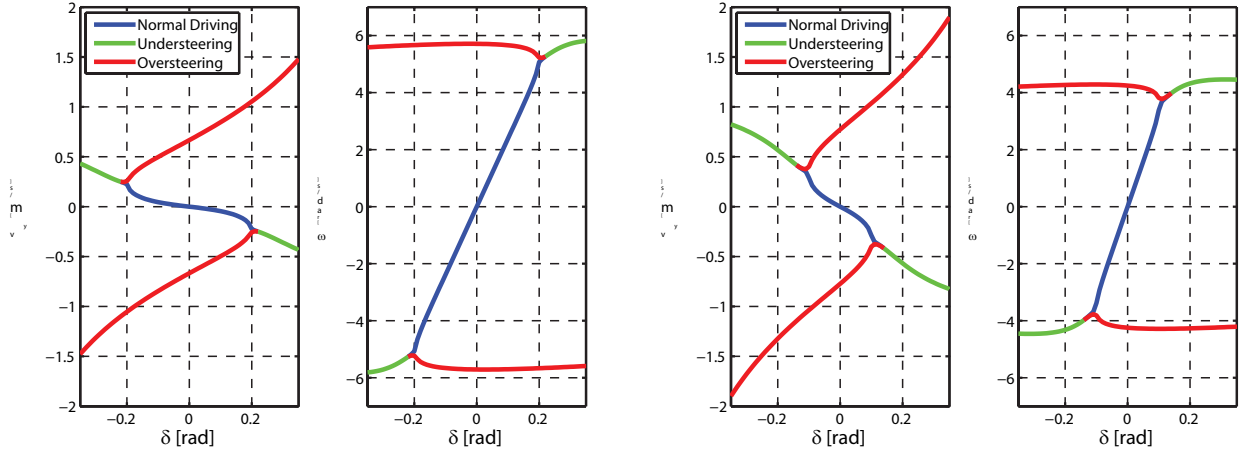


图2: $\bar{v}_x = 1.5$ 米/秒 (左侧图) 和 $\bar{v}_x = 2$ 米/秒 (右侧图) 的稳态速度, 以转向角 $\bar{\delta}$ 为参数。

如图 2 所示, 对于 $\bar{v}_x = 1.5$ 米/秒和 $\bar{v}_x = 2$ 米/秒, 所得的稳定速度被展示出来。所得的稳定速度可以分为三个不同的区域。蓝色曲线表示正常驾驶区域, 其特点是偏航率与转向角呈线性依赖关系, 并且横向速度较小。红色和绿色的另外两个区域分别对应模型中的过度转向点和欠转向点。在欠转向区域 (绿色曲线) 中, 前轮力的饱和律阻止车辆达到更大的偏航率, 因此车辆无法进行更急的转弯。在过度转向区域 (红色曲线) 中, 后轮力完全饱和, 车辆以较高的横向速度行驶, 这通常被称为漂移。

模型在不同区域的动态是不同的, 例如在超转向区域, 当车辆漂移时, 转向角的方向可能与车辆行驶的曲率相反, 这被称为反向转向, 而在正常驾驶模式和转向不足模式下则不会发生。对不同稳定速度点周围的横向速度模型进行稳定性分析也显示了动态的差异: 正常驾驶区域和转向不足区域是稳定的, 而漂移区域是不稳定的 [10]。

从图2可以看出, 当汽车行驶速度更快时, 正常转向区域正在变小。因此, 在轮胎力未饱和的情况下, 最大和最小转向角会随着速度的增加而减小, 同时最大偏航率也会减小。

由于所有速度都是恒定的, 汽车的运动是匀速圆周运动, 其偏航率、半径 R 和曲率 κ 之间的关系是已知的,

$$|\omega| = \frac{v}{R} = v\kappa \quad \text{where} \quad v = \sqrt{v_x^2 + v_y^2}. \quad (3)$$

因此, 所有静止速度点都对应于汽车以恒定半径行驶圆周, 即该模型可以预测沿圆周漂移。

III. 自动驾驶赛车控制

在本文的主要部分, 我们提出了针对遥控车自主赛车任务的两种基于优化的模型, 这些模型基于第二节中描述的模型。首先, 在第三节A部分介绍了分层的两级方法, 其次在第三节B部分介绍了单级方案。然后, 我们简要讨论一种用于避障的高级方法, 该方法在两种控制器中均相同, 见第三节C部分。算法摘要见第三节D部分。

TABLE I: Part of the Path Planning Library

Description	v_x [m/s]	v_y [m/s]	ω [rad/s]	δ [rad]
Left	1.75	0.1185	-2.7895	-0.1000
Straight	1.75	0	0	0
Slight Right	1.75	-0.0486	1.3849	0.05
Hard Right	1.75	-0.3163	4.4350	0.16
Hard Left Drift	1.75	1.1860	-4.8709	0.2

A. Hierarchical Receding Horizon Controller

In our hierarchical control approach, a high-level path planner finds a progress-optimal, feasible trajectory within a finite number of possibilities and for a horizon of N sampling times. This trajectory is then tracked by an MPC controller employing soft-constraints to ensure feasibility of the low-level optimization at all times. This process is repeated in a receding-horizon fashion, hence we call the controller Hierarchical Receding Horizon Controller (HRHC) in what follows. Details on the individual components are given in the following.

1) *Path Planning.*: The purpose of the path planner is to generate a trajectory with maximal progress which is feasible for the car's dynamics. It is based on gridding the stationary velocities of the nonlinear system described in Section II-B. By solving (1) for the stationary lateral velocity points given a set of different longitudinal velocities $\bar{v}_x$ and steering angles $\bar{\delta}$, a library of zero acceleration points is generated. This includes stationary velocities from the normal as well as from the drifting region. Table I shows an excerpt of such a library. The library used in our experiments consists of $N_{\bar{v}} = 95$ stationary points, out of which 26 correspond to drifting equilibria. The stationary points are selected by uniformly gridding the longitudinal velocity v_x between 0.5 and 3.5 m/s, with steps of 0.25 m/s. For each $\bar{v}_x$, about five to nine points are selected in the normal driving region, and up to four drifting points are selected for forward velocities between 1.5 and 2.25 m/s. Note that the library does not change during run-time, hence it can be generated offline.

To generate trajectories for the whole horizon, the stationary points are integrated. This is computationally cheap under the assumption that the stationary velocity can be reached within one time step, since in this case all accelerations are zero. This procedure allows us to generate reference trajectories also for unstable drifting points. The integration of the reasonable stationary velocity points leads to a countable set of possible trajectories with a constant turning radius over the horizon. A visualization of such a set of candidate trajectories is shown in Figure 3.

We now proceed with finding the best trajectory among the candidates obtained by gridding and integration, which is equivalent to solving the following optimization problem:

$$\theta^* \triangleq \max_{j \in 1, \dots, N_{\bar{v}}} \mathcal{P}(X_N^j, Y_N^j), \quad (4a)$$

$$\text{s.t. } x_0^j = x, \quad (4b)$$

$$x_{k+1}^j = f_{km}(x_k^j, \bar{v}^j), \quad k = 1, \dots, N \quad (4c)$$

$$x_k^j \in \mathcal{X}_{track}, \quad k = 1, \dots, N \quad (4d)$$

$$\bar{v}^j \in \mathcal{V}(v), \quad (4e)$$

where $x_k \triangleq (X_k, Y_k, \varphi_k)$ is the position and orientation at time step k generated by integrating the kinematic part of the bicycle model using the stationary velocity $\bar{v}^j \triangleq (\bar{v}_x^j, \bar{v}_y^j, \bar{\omega}^j)$ from the library of stationary points. The integration is denoted by the discrete time version of the model, f_{km} in (4c). The objective (4a) is to find the trajectory with the largest progress on the track, which is measured by the projection operator $\mathcal{P} : \mathbb{R}^2 \rightarrow [0, L]$ that calculates the scalar projection of the position X_k^j, Y_k^j onto the piecewise linear center line parameterized by the arc length $\theta \in [0, L]$, where L is the length of the center

表 I: 路径规划库的一部分

Description	v_x [m/s]	v_y [m/s]	ω [rad/s]	δ [rad]
Left	1.75	0.1185	-2.7895	-0.1000
Straight	1.75	0	0	0
Slight Right	1.75	-0.0486	1.3849	0.05
Hard Right	1.75	-0.3163	4.4350	0.16
Hard Left Drift	1.75	1.1860	-4.8709	0.2

A. Hierarchical Receding Horizon Controller

在我们的分层控制方法中，高级路径规划器在有限数量的可能性和 N 采样时间的视野范围内，找到一个进展最优且可行的轨迹。该轨迹随后由采用软约束的 MPC 控制器进行跟踪，以确保低级优化在任何时候都是可行的。这个过程以递进视野的方式重复进行，因此，我们在下文将该控制器称为分层递进视野控制器（HRHC）。各个组件的详细信息如下所示。

1) *Path Planning.*: 路径规划器的目的是生成一个在车辆动力学可行的情况下具有最大进展的轨迹。它基于对第 II-B 节中描述的非线性系统的静止速度进行网格化。通过对给定不同纵向速度 $\bar{v}_x$ 和转向角 $\bar{\delta}$ 的一组静止侧向速度点求解 (1)，生成零加速度点库。这包括来自正常区域以及漂移区域的静止速度。表 I 显示了该点库的一个摘录。我们实验中使用的点库由 $N_{\bar{v}} = \bar{v}$ 个静止点组成，其中 26 个对应漂移平衡点。静止点的选择通过在 0.5 至 3.5 m/s 之间以 0.25 m/s 的步长均匀网格化纵向速度 v_x 来实现。对于每个 $\bar{v}_x$ ，在正常驾驶区域选择大约五到九个点，对于 1.5 至 2.25 m/s 的前向速度，最多选择四个漂移点。请注意

为了生成整个时间范围内的轨迹，需要对平衡点进行积分。在假设平衡速度可以在一个时间步内达到的情况下，这种方法计算成本低，因为在这种情况下所有加速度都为零。这一过程还允许我们为不稳定的漂移点生成参考轨迹。对合理的平衡速度点进行积分会得到一组在整个时间范围内具有恒定转弯半径的可数轨迹集合。这类候选轨迹集合的可视化如图3所示。

我们现在继续从通过网格化和积分获得的候选轨迹中寻找最佳轨迹，这等同于解决以下优化问题：

$$\theta^* \triangleq \max_{j \in 1, \dots, N_{\bar{v}}} \mathcal{P}(X_N^j, Y_N^j), \quad (4a)$$

$$\text{s.t. } x_0^j = x, \quad (4b)$$

$$x_{k+1}^j = f_{km}(x_k^j, \bar{v}^j), \quad k = 1, \dots, N \quad (4c)$$

$$x_k^j \in \mathcal{X}_{track}, \quad k = 1, \dots, N \quad (4d)$$

$$\bar{v}^j \in \mathcal{V}(v), \quad (4e)$$

其中 $x_k \triangleq (X_k, Y_k, \varphi_k)$ 是在时间步 k 的位置和姿态，通过使用来自静止点库的静止速度 $\bar{v}^j \triangleq (\bar{v}_x^j, \bar{v}_y^j, \bar{\omega}^j)$ 积分单车模型的运动学部分生成。积分由该模型的离散时间版本表示，即 (4c) 中的 f_{km} 。目标 (4a) 是找到在赛道上进展最大的轨迹，该进展通过投影算子 $\mathcal{P}: \mathbb{R}^2 \rightarrow [0, L]$ 来衡量，该算子计算位置 X_k^j, Y_k^j 在由弧长 $\theta \in [0, L]$ 参数化的分段线性中心线上的标量投影，其中 L 是中心线长度。



Fig. 3: Left: Generation of reference trajectories for two different longitudinal velocities (black: slow, green: fast). Right: Solution of the path planner. The red trajectory has the largest progress but leaves the track, so that the orange one is the optimal trajectory maximizing θ_N .

line. All positions of the trajectory x_k^j , $k = 0, \dots, N$, need to lie in the set $\mathcal{X}_{track} \subset \mathbb{R}^2$ (4d) which is the set of all admissible positions inside the track boundaries. Finally, we demand that the stationary velocity $\bar{v}^j$ lies in the set $\mathcal{V} \subset \mathbb{R}^3$ (4e), which depends on the current velocity measurement $v \triangleq (v_x, v_y, \omega)$ and enforces that the planned velocity is reasonably close to the measured velocity, which helps to deal with the assumption that the stationary velocity can be reached within one time step:

$$\mathcal{V}(v) \triangleq \begin{cases} \{(\bar{v}_x, \bar{v}_y, \bar{\omega}) \in \mathbb{R}^3 \mid \rho \geq |v_x - \bar{v}_x|\} & \text{if } |\bar{v}_y| \leq \nu \\ \{(\bar{v}_x, \bar{v}_y, \bar{\omega}) \in \mathbb{R}^3 \mid \sigma_x \geq |v_x - \bar{v}_x|, \sigma_y \geq |v_y - \bar{v}_y|, \sigma_\omega \geq |\omega - \bar{\omega}|\} & \text{otherwise} \end{cases}, \quad (5)$$

where the two cases correspond to selecting non-drift or drift trajectories, respectively, based on the lateral steady state velocity $\bar{v}_y$. The tuning parameters $\nu, \rho, \sigma_x, \sigma_y, \sigma_\omega$ are used to determine the behavior of the path planner in terms of selecting candidate trajectories. Choosing parameters that lead to too loose constraints in (5) result typically in physically impossible trajectories, which can result in crashes. Conversely, constraints that are too tight limit the performance of the controller, as the planned trajectories are too conservative.

Problem (4) is an integer program with the decision variable j and solved by enumeration in practice by checking the discrete positions against local convex inner approximations of the track set, which is reasonable due to the relatively small number of trajectories (about one hundred in our case). The result of the optimization problem is visualized in Figure 3. The path planning allows to efficiently handle drift, as a drifting trajectory is selected if it yields the largest progress.

The path planner has some limitations. Firstly, on one hand, the horizon length has to be quite short, otherwise the planned trajectories become circles and always yield infeasibilities with respect to the track constraints, even if the selected velocity would be well suited for the current state. A horizon which is too short, on the other hand, leads to poor performance as the path planner recognizes a forthcoming curve too late. The second problem is that the whole horizon has the same velocity, which can lead to problems in complicated curve combinations such as chicanes.

2) *Model Predictive Reference Tracking.*: The reference trajectory from the path planner is generated under simplifying assumptions, thus it is not possible to directly apply the controls which correspond to the stationary velocity point. In order to efficiently track the reference the following MPC formulation is

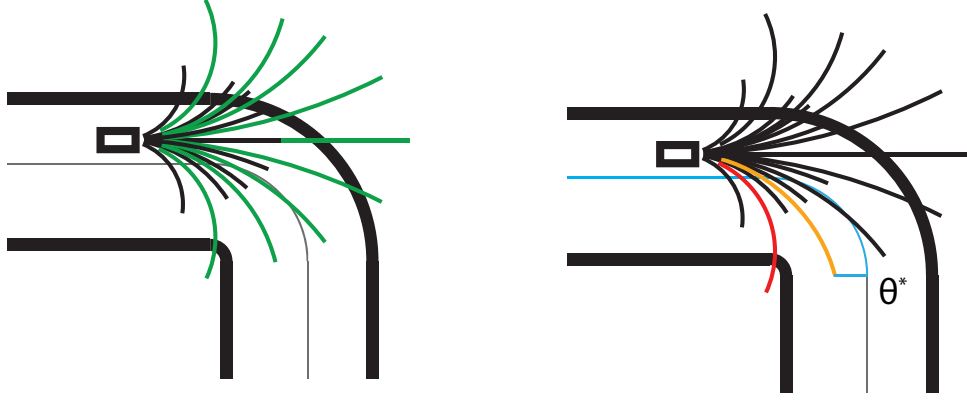


图3: 左: 为两种不同的纵向速度生成参考轨迹 (黑色: 慢, 绿色: 快)。右: 路径规划器的解。红色轨迹进展最大但超出了赛道, 因此橙色轨迹是最大化 θ_N 的最优轨迹。

线路。轨迹 x_k^j , $k = 0, \dots, N$, 的所有位置都必须位于集合 $\mathcal{X}_{track} \subset \mathbb{R}^2$ (4d) 中, 该集合是赛道边界内所有允许位置的集合。最后, 我们要求静止速度 $\bar{v}^j$ 位于集合 $\mathcal{V} \subset \mathbb{R}^3$ (4e) 中, 该集合取决于当前速度测量 $v \triangleq (v_x, v_y, \omega)$, 并确保规划的速度与测量速度保持合理接近, 这有助于处理假设静止速度可以在一个时间步内达到的情况:

$$\mathcal{V}(v) \triangleq \begin{cases} \{(\bar{v}_x, \bar{v}_y, \bar{\omega}) \in \mathbb{R}^3 \mid \rho \geq |v_x - \bar{v}_x|\} & \text{if } |\bar{v}_y| \leq \nu \\ \{(\bar{v}_x, \bar{v}_y, \bar{\omega}) \in \mathbb{R}^3 \mid \sigma_x \geq |v_x - \bar{v}_x|, \sigma_y \geq |v_y - \bar{v}_y|, \sigma_\omega \geq |\omega - \bar{\omega}|\} & \text{otherwise} \end{cases}, \quad (5)$$

这两种情况对应于分别基于横向稳态速度 $\bar{v}_y$ 选择非漂移或漂移轨迹。调节参数 $\nu, \rho, \sigma_x, \sigma_y, \sigma_\omega$ 用于决定路径规划器在选择候选轨迹时的行为。选择导致约束过松的参数 (公式 (5) 中) 通常会生成物理上不可能的轨迹, 这可能导致碰撞。相反, 约束过紧会限制控制器的性能, 因为规划的轨迹过于保守。

问题 (4) 是一个整数规划, 其决策变量为 j , 在实际中通过枚举求解, 即将离散位置与轨道集的局部凸内近似进行比较检查, 这在轨迹数量相对较少 (在我们案例中约为一百条) 的情况下是合理的。优化问题的结果在图 3 中进行了可视化。路径规划能够高效地处理漂移, 因为如果漂移轨迹能够产生最大的前进, 则会选择该轨迹。

路径规划器存在一些限制。首先, 一方面, 视野长度必须相当短, 否则规划的轨迹会变成圆圈, 并且总是会因赛道约束而出现不可行性, 即使所选择的速度非常适合当前状态。另一方面, 视野过短会导致性能较差, 因为路径规划器对即将到来的弯道识别得太晚。第二个问题是整个视野保持相同的速度, 这可能会在复杂弯道组合 (例如蛇形弯) 中引发问题。

2) *Model Predictive Reference Tracking.*: 路径规划器的参考轨迹是在简化假设下生成的, 因此不可能直接应用对应于静止速度点的控制。为了有效地跟踪参考轨迹, 使用了以下MPC公式

used, penalizing the deviation from the reference trajectory using a quadratic function:

$$\min_{x,u,s} \|x_N - x_N^{\text{ref}}\|_P^2 + p\|s_N\|_\infty + \sum_{k=0}^{N-1} \|x_k - x_k^{\text{ref}}\|_Q^2 + q\|s_k\|_\infty + \|u_k - u_k^{\text{ref}}\|_R^2 \quad (6a)$$

$$\text{s.t. } x_0 = x, \quad (6b)$$

$$x_{k+1} = A_k x_k + B_k u_k + g_k, \quad k = 0, \dots, N-1 \quad (6c)$$

$$F_k x_k \leq f_k + s_k, \quad k = 1, \dots, N \quad (6d)$$

$$s_k \geq 0, \quad k = 1, \dots, N \quad (6e)$$

$$\underline{x} \leq x_k \leq \bar{x}, \quad k = 1, \dots, N \quad (6f)$$

$$\underline{u} \leq u_k \leq \bar{u}, \quad k = 0, \dots, N-1 \quad (6g)$$

where $Q \in \mathbb{S}_{++}^6$, $R \in \mathbb{S}_{++}^2$, $P \in \mathbb{S}_{++}^6$ are tuning matrices and $p, q \in \mathbb{R}_{++}$ are penalties for the soft constraints (6d), which are discussed below. To capture as much of the nonlinear model as possible while maintaining computational tractability, the model is linearized around the reference trajectory, cf. (6c). It is necessary to linearize around a trajectory rather than a single operating point, since both position and orientation can vary significantly over the horizon. The reference trajectory is by construction feasible with respect to the track. In order to also guarantee that the trajectory of the optimal MPC solution satisfies track constraints, the X and Y states are limited to stay inside the track if possible. This is achieved by limiting each point in the horizon to lie within two parallel half spaces (6d), which leads to two affine inequality constraints per time step (one for the right and one for the left border), resulting in a convex feasible set. Figure 4 depicts these border constraints (6d), which we formulate as soft constraints in order to avoid infeasibility problems in practice. By using an exact penalty function (an infinity norm in our case), we ensure that the resulting optimization problem is always feasible, and that the original solution of the hard constrained problem is recovered in case it would admit a solution [25], [26]. Finally, the control inputs are limited within their physical constraints, and all other states are limited within reasonable bounds to avoid convergence problems of the solver due to free variables (6f), (6g).

Problem (6) can be reformulated as a convex QP and thus efficiently solved. In this work FORCES [3], [4], is employed to generate tailored C-code that solves instances of (6) where parameters are the initial state x , the matrices defining the dynamics along the reference trajectory (A_k, B_k, g_k) as well as the changing halfspace constraints F_k, f_k .

B. Model Predictive Contouring Control

Contouring control is used in various industrial applications such as machine tools for milling and turning [27] or laser profiling [28]. In these applications, the challenge is to compute inputs that control the movement of the tool along a reference path. The latter is given only in spatial coordinates; the associated velocities, angles, etc. are calculated and imposed by the control algorithm. This is different from tracking controllers in that the controller has more freedom to determine the state trajectories to follow the given path, for example to schedule the velocity, which in tracking is defined by the reference trajectory.

The contour following problem can be formulated in a predictive control framework to incorporate constraints, see e.g. [1]. In this work, we adapt the particular formulation of the model predictive contouring control (MPCC) framework from [2] to obtain a high-performance controller for autonomous racing, which maximizes the travelled distance on the reference path within the prediction horizon. In our experiments, we use the center line as a reference path, but employ it merely as a measure of progress by selecting low weights on the tracking (contouring) error. As a result, the driven trajectory is very similar to those driven by expert drivers. The advantage of this approach is that path planning and path tracking can be combined into one nonlinear optimization problem, which can be solved in real-time by approximating the NLP using local convex QP approximations at each sampling time [21]. The resulting QPs can be

使用二次函数惩罚偏离参考轨迹：

$$\min_{x,u,s} \|x_N - x_N^{\text{ref}}\|_P^2 + p\|s_N\|_\infty + \sum_{k=0}^{N-1} \|x_k - x_k^{\text{ref}}\|_Q^2 + q\|s_k\|_\infty + \|u_k - u_k^{\text{ref}}\|_R^2 \quad (6a)$$

$$\text{s.t. } x_0 = x, \quad (6b)$$

$$x_{k+1} = A_k x_k + B_k u_k + g_k, \quad k = 0, \dots, N-1 \quad (6c)$$

$$F_k x_k \leq f_k + s_k, \quad k = 1, \dots, N \quad (6d)$$

$$s_k \geq 0, \quad k = 1, \dots, N \quad (6e)$$

$$\underline{x} \leq x_k \leq \bar{x}, \quad k = 1, \dots, N \quad (6f)$$

$$\underline{u} \leq u_k \leq \bar{u}, \quad k = 0, \dots, N-1 \quad (6g)$$

其中 $Q \in \mathbb{S}_{++}^6$ 、 $R \in \mathbb{S}_{++}^2$ 、 $P \in \mathbb{S}_{++}^6$ 是调节矩阵， $p, q \in \mathbb{R}_{++}$ 是对软约束 (6d) 的惩罚，如下文所述。为了在保持计算可行性的同时尽可能捕捉非线性模型，模型在参考轨迹附近被线性化，参见 (6c)。有必要围绕轨迹进行线性化，而不是单一操作点，因为在整个预测区间内，位置和方向都可能显著变化。参考轨迹在构造上是符合赛道的。为了保证最优 MPC 解的轨迹也满足赛道约束， X 和 Y 状态在可能情况下被限制保持在赛道内。这是通过限制预测区间内的每个点位于两个平行半空间内 (6d) 实现的，这导致每个时间步有两个仿射不等式约束（一个针对右边界，一个针对左边界），从而形成凸可行集。图 4 描绘了这些边界约束情况。

问题 (6) 可以重新表述为一个凸二次规划 (QP)，因此可以高效地求解。在这项工作中，使用 FORCES [3], [4] 来生成定制的 C 代码，用于求解 (6) 的实例，其中参数是初始状态 x 、定义沿参考轨迹动力学的矩阵 (A_k, B_k, g_k) 以及变化的半空间约束 F_k, f_k 。

B. Model Predictive Contouring Control

轮廓控制用于各种工业应用，例如用于铣削和车削的机床[27]或激光轮廓加工[28]。在这些应用中，挑战在于计算输入以控制工具沿参考路径的运动。参考路径仅以空间坐标给出；相关的速度、角度等由控制算法计算并施加。这与跟踪控制器不同，因为控制器在确定状态轨迹以遵循给定路径方面有更多自由，例如在跟踪中速度由参考轨迹定义，而轮廓控制器可以对速度进行调度。

轮廓跟随问题可以在预测控制框架下进行表述以纳入约束，参见例如[1]。在本工作中，我们采用[2]中模型预测轮廓控制 (MPCC) 框架的特定表述，以获得用于自主赛车的高性能控制器，该控制器在预测范围内最大化在参考路径上的行驶距离。在我们的实验中，我们使用中心线作为参考路径，但仅通过在跟踪（轮廓）误差上选择较低权重，将其作为进度的衡量方式。因此，行驶的轨迹与专家司机驾驶的轨迹非常相似。这种方法的优势在于路径规划和路径跟踪可以结合为一个非线性优化问题，可以通过在每个采样时间使用局部凸二次规划 (QP) 逼近非线性规划 (NLP) 来实时求解[21]。得到的QP可以被

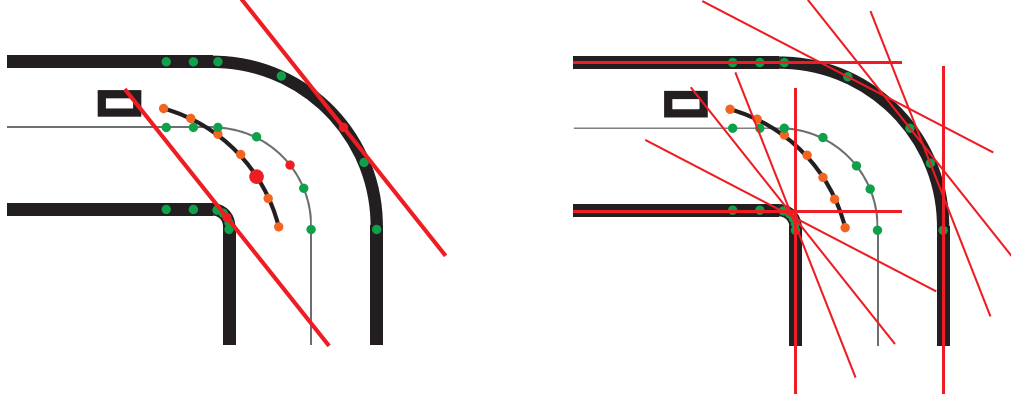


Fig. 4: Left: Halfspace constraints (red lines) of one point in the horizon (large red dot). Right: Resulting halfspaces (red lines) of the whole horizon, approximating the track. The green points on the center line correspond to the closest point on the discrete center line, for each point in the horizon. The points on the borders are the projection of the center line points on the borders.

formulated with multistage structure, which is exploited by FORCES [3], [4] to obtain solution times in the range of a few milliseconds.

Before posing the MPCC problem, a few preliminaries are introduced such as the parameterization of the reference path and the definition of useful error measures.

1) *Parameterization of Reference Trajectory.*: The reference path is parameterized by its arc length $\theta \in [0, L]$ using third order spline polynomials, where L is the total length. The splines are obtained by an offline fitting of the center line. Using this parameterization, we can obtain any point $X^{\text{ref}}(\theta)$, $Y^{\text{ref}}(\theta)$ on the center line by evaluating a third order polynomial for its argument θ . The angle of the tangent to the path at the reference point with respect to the X -axis,

$$\Phi(\theta) \triangleq \arctan \left\{ \frac{\partial Y^{\text{ref}}(\theta)}{\partial X^{\text{ref}}(\theta)} \right\}, \quad (7)$$

is also readily available. This parameterization leads to an accurate interpolation within the known points of the reference path, and it is more accurate than the piece-wise linear parameterization employed for the HRHC in Section III-A, which only uses a linear interpolation between the points.

2) *Error measures.*: In order to formulate the MPCC problem, error measures are needed that define the deviation of the car's current position X, Y from the desired reference point $X^{\text{ref}}(\theta)$, $Y^{\text{ref}}(\theta)$. We use the same definitions as in [2], but give another derivation here. Let $\mathcal{P} : \mathbb{R}^2 \rightarrow [0, L]$ be a projection operator on the reference trajectory defined by

$$\mathcal{P}(X, Y) \triangleq \arg \min_{\theta} (X - X^{\text{ref}}(\theta))^2 + (Y - Y^{\text{ref}}(\theta))^2. \quad (8)$$

For brevity, we define $\theta_{\mathcal{P}} \triangleq \mathcal{P}(X, Y)$. The orthogonal distance of the car from the reference path is then given by the *contouring error*

$$e^c(X, Y, \theta_{\mathcal{P}}) \triangleq \sin(\Phi(\theta_{\mathcal{P}})) (X - X^{\text{ref}}(\theta_{\mathcal{P}})) - \cos(\Phi(\theta_{\mathcal{P}})) (Y - Y^{\text{ref}}(\theta_{\mathcal{P}})), \quad (9)$$

where $\Phi(\cdot)$ is defined in (7). The contouring error is depicted in the left picture in Figure 5.

The projection operator (8) is not well suited for use within online optimization algorithms, as it resembles an optimization problem itself. Thus an approximation $\theta_{\mathcal{A}}$ of $\theta_{\mathcal{P}}$ is introduced, which is an independent variable determined by the controller. For this approximation to be useful, it is necessary to link $\theta_{\mathcal{A}}$ to $\theta_{\mathcal{P}}$ via the *lag error*

$$e^l(X, Y, \theta_{\mathcal{A}}) \triangleq |\theta_{\mathcal{A}} - \theta_{\mathcal{P}}|, \quad (10)$$

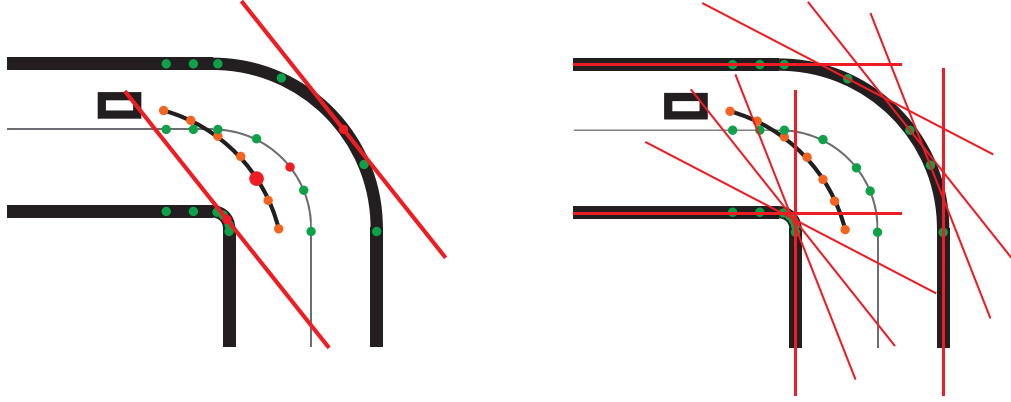


图4: 左: 水平线上某一点(大红点)的半空间约束(红线)。右: 整个水平线的生成半空间(红线), 近似轨迹。中心线上绿色的点对应于水平线上每个点在离散中心线上最接近的点。边界上的点是中心线点在边界上的投影。

采用多级结构制定, FORCES [3], [4] 利用该结构获得几毫秒范围内的解算时间。

在提出 MPCC 问题之前, 先介绍一些预备知识, 例如参考路径的参数化以及有用误差度量的定义。

1) *Parameterization of Reference Trajectory.*: 参考路径通过其弧长 $\theta \in [0, L]$ 使用三阶样条多项式进行参数化, 其中 L 是总长度。样条是通过对中心线的离线拟合获得的。利用这种参数化, 我们可以通过对参数 θ 计算三阶多项式来获得中心线上任意点 $X^{\text{ref}}(\theta)$, $Y^{\text{ref}}(\theta)$ 。路径在参考点处的切线与 X 轴的夹角,

$$\Phi(\theta) \triangleq \arctan \left\{ \frac{\partial Y^{\text{ref}}(\theta)}{\partial X^{\text{ref}}(\theta)} \right\}, \quad (7)$$

也很容易获取。这种参数化在参考路径的已知点之间实现了精确的插值, 其精度比在第 III-A 节中用于 HRHC 的分段线性参数化更高, 后者仅在点之间使用线性插值。

2) *Error measures.*: 为了构建 MPCC 问题, 需要误差度量来定义汽车当前位置 X, Y 与期望参考点 $X^{\text{ref}}(\theta)$, $Y^{\text{ref}}(\theta)$ 之间的偏差。我们使用与 [2] 中相同的定义, 但在此给出另一种推导。设 $\mathcal{P}: \mathbb{R}^2 \rightarrow [0, L]$ 为参考轨迹上的投影算子, 定义如下

$$\mathcal{P}(X, Y) \triangleq \arg \min_{\theta} (X - X^{\text{ref}}(\theta))^2 + (Y - Y^{\text{ref}}(\theta))^2. \quad (8)$$

为了简洁, 我们定义 $\theta_{\mathcal{P}} \triangleq \mathcal{P}(X, Y)$ 。然后汽车到参考路径的正交距离由 *contouring error* 给出

$$e^c(X, Y, \theta_{\mathcal{P}}) \triangleq \sin(\Phi(\theta_{\mathcal{P}})) (X - X^{\text{ref}}(\theta_{\mathcal{P}})) - \cos(\Phi(\theta_{\mathcal{P}})) (Y - Y^{\text{ref}}(\theta_{\mathcal{P}})), \quad (9)$$

其中 $\Phi(\cdot)$ 在 (7) 中定义。轮廓误差如图 5 左侧所示。

投影算子 (8) 不太适合在线优化算法中使用, 因为它本身类似于一个优化问题。因此, 引入了 $\theta_{\mathcal{P}}$ 的一个近似 θ_A , 它是由控制器确定的独立变量。为了使这个近似有用, 有必要通过 *lag error* 将 θ_A 与 $\theta_{\mathcal{P}}$ 关联起来。

$$e^l(X, Y, \theta_A) \triangleq |\theta_A - \theta_{\mathcal{P}}|, \quad (10)$$

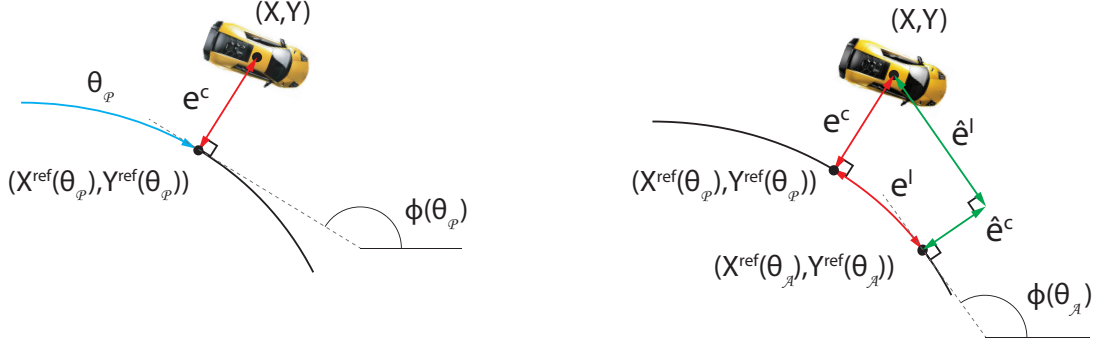


Fig. 5: Contouring error e^c (left) and lag error e^l (right) with linear approximations $\hat{e}^c$ and $\hat{e}^l$.

which measures the quality of the approximation. The lag error is depicted in the right picture in Figure 5 (red segment on reference path).

In order to be independent of the projection operator (8), the contouring (9) and the lag error (10) can be approximated as a function of the position X, Y and the approximate projection θ_A , which are variables that can be controlled by the MPCC controller:

$$e^c \approx \hat{e}^c(X, Y, \theta_A) \triangleq \sin(\Phi(\theta_A)) (X - X^{\text{ref}}(\theta_A)) - \cos(\Phi(\theta_A)) (Y - Y^{\text{ref}}(\theta_A)) , \quad (11a)$$

$$e^l \approx \hat{e}^l(X, Y, \theta_A) \triangleq -\cos(\Phi(\theta_A)) (X - X^{\text{ref}}(\theta_A)) - \sin(\Phi(\theta_A)) (Y - Y^{\text{ref}}(\theta_A)) . \quad (11b)$$

The approximate contouring error $\hat{e}^c$ (11a) and the approximate lag error $\hat{e}^l$ (11b) are defined as the orthogonal and tangential component of the error between $X^{\text{ref}}(\theta_A), Y^{\text{ref}}(\theta_A)$ and the position X, Y , see the right picture in Figure 5.

3) *MPCC problem.*: With these error measures in place, we now formulate the model predictive contouring control problem for autonomous racing. The objective is a trade-off between the quality of path-following (low contouring error) and the amount of progress achieved over a finite horizon of N sampling times, subject to model dynamics, track- and input constraints:

$$\min \sum_{k=1}^N \{ \|e_k^c(X_k, Y_k, \theta_P)\|_{q_c}^2 \} - \gamma \theta_{P,N} \quad (12a)$$

$$\text{s.t. } x_0 = x , \quad (12b)$$

$$x_{k+1} = f(x_k, u_k) , \quad k = 0, \dots, N-1 \quad (12c)$$

$$F_k x_k \leq f_k , \quad k = 1, \dots, N \quad (12d)$$

$$\underline{x} \leq x_k \leq \bar{x} , \quad k = 1, \dots, N \quad (12e)$$

$$\underline{u} \leq u_k \leq \bar{u} , \quad k = 0, \dots, N-1 \quad (12f)$$

where X_k, Y_k is the position of the car at time step k determined by the nonlinear model f in (12c), which is the discrete-time version of (1) with piece-wise constant control inputs. The contouring error $e_k^c(X_k, Y_k, \theta_P)$ in the objective (12a) is defined in (9), and $\theta_{P,k}$ is the associated path parameter such that $X^{\text{ref}}(\theta_{P,k}), Y^{\text{ref}}(\theta_{P,k})$ is the orthogonal projection of X_k, Y_k onto the reference path. The two objectives (maximum progress and tight path following) are traded off by the weights $\gamma \in \mathbb{R}_{++}$ and $q_c \in \mathbb{R}_{++}$, respectively. Constraints (12d) are the parallel half space constraints for containing the position in the allowed corridor as described in Section III-A2, while (12e), (12f) corresponds to (6f), (6g), limiting states and inputs to physically admissible values.

Due to the implicit dependency of the objective (12a) on the projection operator (8), optimization problem (12) is a bi-level NLP, which is too complex to solve in real-time. Thus the idea is to use the approximate projection $\theta_{A,k}$ instead of $\theta_{P,k}$ as introduced in Section III-B2, and to control the

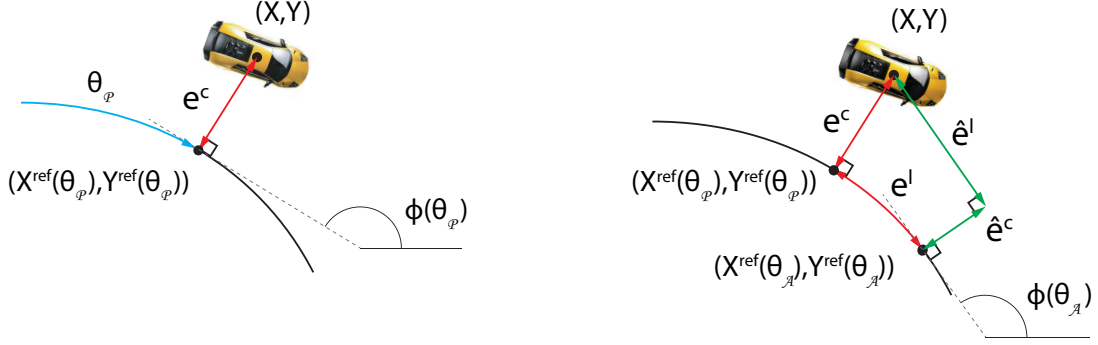


图5: 轮廓误差 e^c (左) 和滞后误差 e^l (右) 与线性逼近 $\hat{e}^c$ 和 $\hat{e}^l$ 。

它衡量逼近的质量。滞后误差如图5右图所示（参考路径上的红色部分）。

为了独立于投影算子 (8)，轮廓 (9) 和滞后误差 (10) 可以近似为位置 X, Y 和近似投影 θ_A 的函数，这些都是可以由 MPCC 控制器控制的变量：

$$e^c \approx \hat{e}^c(X, Y, \theta_A) \triangleq \sin(\Phi(\theta_A)) (X - X^{\text{ref}}(\theta_A)) - \cos(\Phi(\theta_A)) (Y - Y^{\text{ref}}(\theta_A)), \quad (11a)$$

$$e^l \approx \hat{e}^l(X, Y, \theta_A) \triangleq -\cos(\Phi(\theta_A)) (X - X^{\text{ref}}(\theta_A)) - \sin(\Phi(\theta_A)) (Y - Y^{\text{ref}}(\theta_A)). \quad (11b)$$

近似轮廓误差 $\hat{e}^c$ (11a) 和近似滞后误差 $\hat{e}^l$ (11b) 定义为 $X^{\text{ref}}(\theta_A), Y^{\text{ref}}(\theta_A)$ 与位置 X, Y 之间误差的正交分量和平行分量，见图 5 右图。

3) *MPCC problem.*: 在这些误差度量到位之后，我们现在为自主赛车制定模型预测轮廓控制问题。其目标是在路径跟随的质量（低轮廓误差）与在有限的 N 个采样时间内取得的进度量之间进行权衡，同时受模型动力学、赛道和输入约束的制约：

$$\min \sum_{k=1}^N \{ \|e_k^c(X_k, Y_k, \theta_P)\|_{q_c}^2 \} - \gamma \theta_{P,N} \quad (12a)$$

$$\text{s.t. } x_0 = x, \quad (12b)$$

$$x_{k+1} = f(x_k, u_k), \quad k = 0, \dots, N-1 \quad (12c)$$

$$F_k x_k \leq f_k, \quad k = 1, \dots, N \quad (12d)$$

$$\underline{x} \leq x_k \leq \bar{x}, \quad k = 1, \dots, N \quad (12e)$$

$$\underline{u} \leq u_k \leq \bar{u}, \quad k = 0, \dots, N-1 \quad (12f)$$

其中 X_k, Y_k 是在时间步 k 车辆的位置，由非线性模型 f 在 (12c) 中确定，(12c) 是 (1) 的离散时间版本，并具有分段恒定的控制输入。目标函数 (12a) 中的轮廓误差 $e_k^c(X_k, Y_k, \theta_P)$ 在 (9) 中定义， $\theta_{P,k}$ 是相关的路径参数，使得 $X^{\text{ref}}(\theta_{P,k}), Y^{\text{ref}}(\theta_{P,k})$ 是 X_k, Y_k 在参考路径上的正交投影。两个目标（最大进度和严格的路径跟随）分别通过权重 $\gamma \in \mathbb{R}_{++}$ 和 $q_c \in \mathbb{R}_{++}$ 进行权衡。约束条件 (12d) 是用于将位置限制在允许走廊内的平行半空间约束，如第三节 III-A2 所述，而 (12e)、(12f) 对应于 (6f)、(6g)，将状态和输入限制在物理允许的值范围内。

由于目标 (12a) 对投影算子 (8) 存在隐式依赖，优化问题 (12) 是一个双层非线性规划 (NLP)，其复杂性过高，无法实时求解。因此，想法是使用第 III-B2 节中引入的近似投影 $\theta_{A,k}$ 代替 $\theta_{P,k}$ ，并控制

approximation quality by adding a cost on the lag error (10) to the objective. In order to allow for forming the lag error at each time step in the prediction horizon, it is necessary to introduce an integrator state with dynamics $\theta_{\mathcal{A},k+1} = \theta_{\mathcal{A},k} + v_k/T_s$, where v_k can be interpreted as the *projected velocity*, and $\theta_{\mathcal{A},k}$ as the *state of progress* at time k , respectively. This approximation reduces (12) to an optimal control problem in form of an NLP that is amenable for a real-time implementation:

$$\min \sum_{k=1}^N \|\hat{e}_k^c(X_k, Y_k, \theta_{\mathcal{A},k})\|_{q_c}^2 + \|\hat{e}_k^l(X_k, Y_k, \theta_{\mathcal{A},k})\|_{q_l}^2 - \gamma v_k T_s + \|\Delta u_k\|_{R_u}^2 + \|\Delta v_k\|_{R_v}^2 - \gamma v_0 T_s \quad (13a)$$

$$\text{s.t. (12b), } \theta_0 = \theta, \quad (13b)$$

$$(12c), \quad \theta_{\mathcal{A},k+1} = \theta_{\mathcal{A},k} + \frac{v_k}{T_s}, \quad k = 0, \dots, N-1 \quad (13c)$$

$$(12d), \quad (12e), \quad 0 \leq \theta_k \leq L, \quad k = 1, \dots, N \quad (13d)$$

$$(12f), \quad 0 \leq v_k \leq \bar{v}, \quad k = 0, \dots, N \quad (13e)$$

where $\Delta u_k \triangleq u_k - u_{k-1}$ and $\Delta v_k \triangleq v_k - v_{k-1}$. Note that the approximate contouring error $\hat{e}_k^c(X_k, Y_k, \theta_{\mathcal{A},k})$ (11a) is used in the objective (13a). Furthermore, we have replaced the maximization of the final progress measure, $\theta_{\mathcal{P},N}$, by $\sum_{k=0}^{N-1} v_k T_s$, which is equivalent if the approximation is accurate. Sensible lower and upper bounds on θ_k and v_k are imposed to avoid spurious solutions of the NLP, with $\bar{v}$ denoting the largest possible progress per sampling time. The cost on the lag error $\hat{e}_k^l(X_k, Y_k, \theta_{\mathcal{A},k})$ in (13a) links the state of progress to the dynamics of the car. To ensure an accurate progress approximation and thus a strong coupling between the cost function and the car model, the weight on the lag error $q_l \in \mathbb{R}_{++}$ is chosen high as suggested in [2]. Furthermore, a cost term on the rate of change of the inputs is added to the objective (13a) in order to penalize fast changing controls, which helps to obtain smooth control inputs in order to prevent amplifying unmodeled dynamics.

4) *Solving the MPCC problem.*: In order to solve the nonlinear optimal control problem (13) in real-time, local convex approximations of (13) in form of the following QPs are built at each sampling time by linearization of nonlinear terms:

$$\min_{x,u,\theta,v,s} \sum_{k=1}^N \begin{bmatrix} x_k \\ \theta_{\mathcal{A},k} \end{bmatrix}^T \Gamma_k \begin{bmatrix} x_k \\ \theta_{\mathcal{A},k} \end{bmatrix} + c_k^T \begin{bmatrix} x_k \\ \theta_{\mathcal{A},k} \end{bmatrix} - \gamma v_k T_s + \begin{bmatrix} \Delta u_k \\ \Delta v_k \end{bmatrix}^T R \begin{bmatrix} \Delta u_k \\ \Delta v_k \end{bmatrix} + q \|s_k\|_{\infty} - \gamma v_0 T_s \quad (14a)$$

$$\text{s.t. } x_0 = x, \quad \theta_{\mathcal{A},0} = \theta, \quad (14b)$$

$$x_{k+1} = A_k x_k + B_k u_k + g_k, \quad k = 0, \dots, N-1 \quad (14c)$$

$$\theta_{\mathcal{A},k+1} = \theta_{\mathcal{A},k} + \frac{v_k}{T_s}, \quad k = 0, \dots, N-1 \quad (14d)$$

$$F_k x_k \leq f_k + s_k, \quad k = 1, \dots, N \quad (14e)$$

$$s_k \geq 0, \quad k = 1, \dots, N \quad (14f)$$

$$\underline{x} \leq x_k \leq \bar{x}, \quad 0 \leq \theta_{\mathcal{A},k} \leq L, \quad k = 1, \dots, N \quad (14g)$$

$$\underline{u} \leq u_k \leq \bar{u}, \quad 0 \leq v_k \leq \bar{v}, \quad k = 0, \dots, N \quad (14h)$$

where $\Gamma_k \in \mathbb{S}_+^7$ is formed by the quadratic part of the linearized contouring and lag error cost function from (13a) and $c_k \in \mathbb{R}^7$ stems from the linear part, respectively. In order to keep the linearization error small, we use an LTV approximation of the dynamics (1) as well as of the contouring and lag errors (11). Each nonlinear function is linearized around the output of the last QP iteration shifted by one stage. The measurement $x = x_0$ is used as the first linearization point, and the last input of the previous iteration is kept constant to generate a new last input. The linearization point for the terminal state x_N is calculated by simulating the nonlinear model for one time step. The track or corridor constraints (14e) are formulated by two half space constraints tangential to the track per time step, as described in Section III-A. These

通过在目标函数中增加滞后误差的成本 (10) 来提高近似质量。为了在预测水平的每个时间步形成滞后误差，有必要引入一个具有动态 $\theta_{\mathcal{A},k+1} = \theta_{\mathcal{A},k} + v_k/T_s$ 的积分状态，其中 v_k 可以解释为 *projected velocity*，而 $\theta_{\mathcal{A},k}$ 可以分别解释为时间 k 的 *state of progress*。这种近似将 (12) 简化为一种以 NLP 形式表达的最优控制问题，适合实时实现：

$$\min \sum_{k=1}^N \|\hat{e}_k^c(X_k, Y_k, \theta_{\mathcal{A},k})\|_{q_c}^2 + \|\hat{e}_k^l(X_k, Y_k, \theta_{\mathcal{A},k})\|_{q_l}^2 - \gamma v_k T_s + \|\Delta u_k\|_{R_u}^2 + \|\Delta v_k\|_{R_v}^2 - \gamma v_0 T_s \quad (13a)$$

$$\text{s.t. (12b), } \theta_0 = \theta, \quad (13b)$$

$$(12c), \quad \theta_{\mathcal{A},k+1} = \theta_{\mathcal{A},k} + \frac{v_k}{T_s}, \quad k = 0, \dots, N-1 \quad (13c)$$

$$(12d), (12e), \quad 0 \leq \theta_k \leq L, \quad k = 1, \dots, N \quad (13d)$$

$$(12f), \quad 0 \leq v_k \leq \bar{v}, \quad k = 0, \dots, N \quad (13e)$$

其中 $\Delta u_k \triangleq u_k - u_{k-1}$ 和 $\Delta v_k \triangleq v_k - v_{k-1}$ 。注意，在目标函数 (13a) 中使用了近似轮廓误差 $\hat{e}_k^c(X_k, Y_k, \theta_{\mathcal{A},k})$ (11a)。此外，我们已经将最终进度度量 $\theta_{\mathcal{P},N}$ 的最大化替换为 $\sum_{k=0}^{N-1} v_k T_s$ ，如果近似足够准确，这两者是等价的。在 θ_k 和 v_k 上施加合理的下限和上限，以避免 NLP 的虚假解，其中 $\bar{v}$ 表示每个采样时间内可能的最大进度。(13a) 中对滞后误差 $\hat{e}_k^l(X_k, Y_k, \theta_{\mathcal{A},k})$ 的成本将进度状态与车辆动力学联系起来。为了确保进度近似的准确性，从而在成本函数与车辆模型之间建立强耦合，滞后误差 $q_l \in \mathbb{R}_{++}$ 的权重选取较高，如文献 [2] 所建议的。此外，目标函数 (13a) 中还增加了对输入变化率的成本项，以惩罚快速变化的控制，这有助于获得平滑的控制输入，从而防止未建模动力学的放大。

4) *Solving the MPCC problem.*: 为了实时求解非线性最优控制问题 (13)，通过对非线性项进行线性化，在每个采样时间构建形式如下的局部凸近似 QPs:

$$\min_{x,u,\theta,v,s} \sum_{k=1}^N \begin{bmatrix} x_k \\ \theta_{\mathcal{A},k} \end{bmatrix}^T \Gamma_k \begin{bmatrix} x_k \\ \theta_{\mathcal{A},k} \end{bmatrix} + c_k^T \begin{bmatrix} x_k \\ \theta_{\mathcal{A},k} \end{bmatrix} - \gamma v_k T_s + \begin{bmatrix} \Delta u_k \\ \Delta v_k \end{bmatrix}^T R \begin{bmatrix} \Delta u_k \\ \Delta v_k \end{bmatrix} + q \|s_k\|_{\infty} - \gamma v_0 T_s \quad (14a)$$

$$\text{s.t. } x_0 = x, \quad \theta_{\mathcal{A},0} = \theta, \quad (14b)$$

$$x_{k+1} = A_k x_k + B_k u_k + g_k, \quad k = 0, \dots, N-1 \quad (14c)$$

$$\theta_{\mathcal{A},k+1} = \theta_{\mathcal{A},k} + \frac{v_k}{T_s}, \quad k = 0, \dots, N-1 \quad (14d)$$

$$F_k x_k \leq f_k + s_k, \quad k = 1, \dots, N \quad (14e)$$

$$s_k \geq 0, \quad k = 1, \dots, N \quad (14f)$$

$$\underline{x} \leq x_k \leq \bar{x}, \quad 0 \leq \theta_{\mathcal{A},k} \leq L, \quad k = 1, \dots, N \quad (14g)$$

$$\underline{u} \leq u_k \leq \bar{u}, \quad 0 \leq v_k \leq \bar{v}, \quad k = 0, \dots, N \quad (14h)$$

其中 $\Gamma_k \in \mathbb{S}_+^7$ 由线性化轮廓跟踪和滞后误差代价函数的二次部分（来自公式 (13a)）形成， $c_k \in \mathbb{R}^7$ 分别来自线性部分。为了保持线性化误差较小，我们使用系统动力学 (1) 以及轮廓跟踪和滞后误差 (11) 的时变线性近似。每个非线性函数在上一次 QP 迭代输出向前移一阶段处进行线性化。测量值 $x = x_0$ 用作第一个线性化点，并且保持前一次迭代的最后输入恒定以生成新的最后输入。终端状态 x_N 的线性化点通过对非线性模型进行一个时间步的仿真计算得出。轨道或走廊约束 (14e) 通过每个时间步切向于轨道的两个半空间约束来表达，如第 III-A 节所述。这些

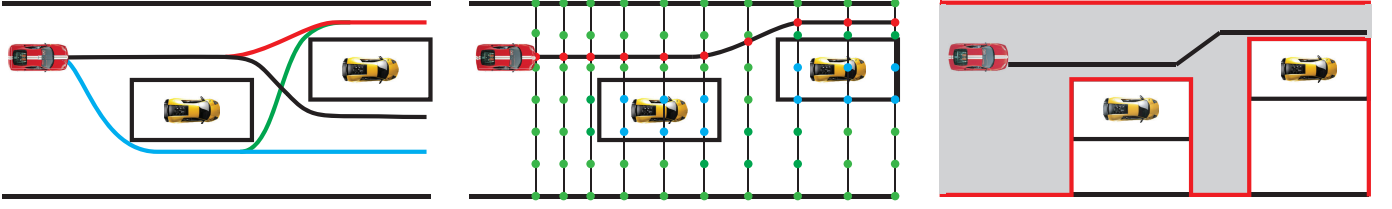


Fig. 6: Left: Combinatorial nature of the overtaking problem, as it is possible to overtake each opponent on the left or the right side. Center: Spatial-temporal grid for the shortest path problem solved by dynamic programming, where it is not allowed to visit the blue grid points. Right: Optimal path of the DP (in black) and the resulting corridor, which is issued to the HRHC (Section III-A) or MPCC (Section III-B).

constraints are formulated as soft constraints, with slack variables $s_k \in \mathbb{R}^2$ and a corresponding infinity-norm penalty in the objective (14a) weighted by $q \in \mathbb{R}_{++}$, which is chosen quite high to recover the behavior of the hard constrained problem whenever possible [25], [26].

Note that the described re-linearization scheme is essentially the basic real-time iteration from [21], where a nonlinear continuous-time optimal control problem is solved using a multiple shooting technique, with an exponential map as integrator, and in every time step only one step of a sequential QP (SQP) scheme is performed. Hence the convergence properties that are known for the real-time iteration [29] hold also for the MPCC problem (13).

The local QP (14) is solved using FORCES [3], [4]. Due to the rate costs, the problem has to be lifted to obtain the multistage structure, for which FORCES has been designed, by introducing a copy of the previous control input at each stage.

C. Obstacle Avoidance

The two controllers presented in this section plan a path within the track, or a corridor, by taking into account constraints (4d), (6f) or (14e) on the position of the car. Thus it is possible to include obstacle avoidance by adapting this corridor online depending on the current constellation of opponents. Generating the optimal corridor based on the position of the obstacles is in general hard, because of the fundamental problem that the decision on which side to overtake is non-convex and of combinatorial nature if more than one opponent is involved. This situation is depicted in Figure 6 in the left picture.

In this work, we employ a high level path planner based on dynamic programming (DP) to decide on which side to overtake the obstacles. Based on the result of the DP, the adapted corridor is given to the controllers of Section III-A or Section III-B. The high level path planner solves a shortest path problem [30] on a spatial-temporal grid, cf. Figure 6. To incorporate the planned path of the lower level controller into the DP, the grid is built up based on the last output of the path planner in the case of the HRHC or the last QP solution in the case of the MPCC. The DP minimizes the travelled distance, and additionally the deviation from the last plan of the lower level controller, such that the DP path is close to the state prediction of the lower level controller. This ensures that the linearizations stay approximately valid most of the time. Based on the optimal trajectory from the DP, the corridor constraints can easily be identified, see the right picture in Figure 6.

Due to space restrictions, we do not go further into detail, but many references exist that solve the corridor problem with different approaches. For example, a similar problem had to be solved during the DARPA Urban challenge where the goal was autonomous driving in city traffic. In [7] several approaches to this problem are presented, for example based on model predictive trajectory generation algorithms [31], modified A^* -algorithms [32] or RRTs [33].

D. Summary

In this main part of the paper, we have presented two approaches for automatic racing. Both approaches are model-based and maximize progress within a given time horizon. They directly incorporate track

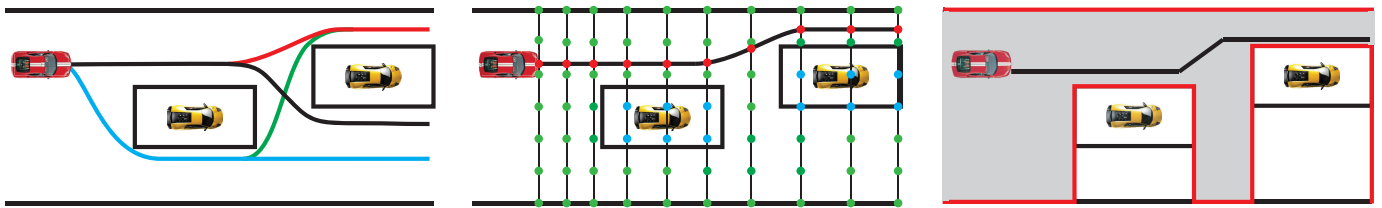


图6: 左图: 超车问题的组合性质, 因为可以从左侧或右侧超越每个对手。中图: 由动态规划求解的最短路径问题的时空网格, 其中不允许访问蓝色网格点。右图: 动态规划的最优路径 (黑色) 及生成的通道, 该通道将用于HRHC (第III-A节) 或MPCC (第III-B节)。

约束被制定为软约束, 带有松弛变量 $s_k \in \mathbb{R}^2$, 并在目标函数 (14a) 中对应以无穷范数惩罚, 由 $q \in \mathbb{R}_{++}$ 加权, 该权重选择得相当高, 以便在可能的情况下恢复硬约束问题的行为 [25], [26]。

请注意, 所描述的重新线性化方案本质上是来自 [21] 的基本实时迭代, 其中非线性连续时间最优控制问题使用多段射击技术求解, 并以指数映射作为积分器, 在每个时间步中仅执行顺序二次规划 (SQP) 方案的一步。因此, 已知的实时迭代 [29] 的收敛性质也适用于 MPCC 问题 (13)。

局部 QP (14) 使用 FORCES [3], [4] 进行求解。由于速率成本, 该问题必须被提升以获得多阶段结构, 这是 FORCES 所设计的, 通过在每个阶段引入之前控制输入的副本实现。

C. Obstacle Avoidance

本节介绍的两个控制器通过考虑车辆位置的约束 (4d)、(6f) 或 (14e), 在赛道或通道内规划路径。因此, 可以通过根据当前对手的排列情况在线调整该通道来实现障碍物避免。基于障碍物位置生成最优通道通常是困难的, 因为根本性问题在于, 如果涉及多个对手, 决定超车位置是非凸的并且具有组合性质。左图的情形在图6中展示。

在这项工作中, 我们使用基于动态图规划 (DP) 的高层路径规划器来决定绕过障碍物的侧面。根据DP的结果, 适应后的走廊会提供给第III-A节或第III-B节的控制器。高层路径规划器在时空网格上解决最短路径问题[30], 参见图6。为了将低层控制器规划的路径纳入DP, 网格是基于HRHC情况下路径规划器的上一次输出或MPCC情况下的上一次QP解构建的。DP最小化行驶距离, 同时也最小化与低层控制器上一次规划的偏差, 从而使DP路径接近低层控制器的状态预测。这确保了线性化在大多数时间内都近似有效。基于DP的最优轨迹, 可以轻松识别走廊约束, 见图6右图。

由于空间限制, 我们不会详细展开, 但存在许多参考文献用不同的方法解决走廊问题。例如, 在 DARPA 城市挑战赛期间, 需要解决一个类似的问题, 其目标是在城市交通中实现自动驾驶。在文献 [7] 中提出了几种解决该方法的方法, 例如基于模型预测轨迹生成算法 [31]、修改的 A^* 算法 [32] 或 RRTs [33]。

D. Summary

在论文的这一主要部分中, 我们提出了两种自动赛车的方法。两种方法都是基于模型的, 并在给定的时间范围内最大化进度。它们直接融入了赛道

and obstacle constraints by using two parallel affine inequalities (slabs) representing the feasible set of positions at each time. The major difference between the approaches is that the first (cf. Section III-A and Algorithm Ia) is a two-level approach with a separate path planning mechanism combined with a reference tracking MPC controller, while the second approach (cf. Section III-B and Algorithm Ib) formulates both tasks (path planning and path tracking) into one nonlinear optimal control problem based on model predictive contouring control.

Algorithm 1 HRHC and MPCC algorithm

(Ia) HRHC

- 1: get current position and velocities
- 2: compute delay compensation
- 3: **function** BORDERADJUSTMENT
- 4: shift old planned path by one stage
- 5: get position of opposing cars
- 6: run DP and get new borders (Sec. III-C)
- 7: **end function**
- 8: run path planner (4)
- 9: linearize and discretize the model, i.e. build problem (6)
- 10: solve QP using FORCES (6)
- 11: send first control at the end of time slot
- 12: **go to** step 1

(Ib) MPCC

- 1: get current position and velocities
 - 2: compute delay compensation
 - 3: augment old QP output (Section III-B4)
 - 4: **function** BORDERADJUSTMENT
 - 5: get position of opposing cars
 - 6: run DP and get new borders (Sec. III-C)
 - 7: **end function**
 - 8: linearize and discretize the model and the cost function, i.e. build problem (14)
 - 9: solve QP using FORCES (14)
 - 10: send first control at the end of time slot
 - 11: **go to** step 1
-

Both controllers send the new control input at the end of the sampling period. In order to compensate for this fixed delay, the nonlinear model is simulated forward for this time interval at the beginning of the algorithm, see line 2 of Algorithm Ia and Ib. The nonlinear model is integrated using a second-order Runge-Kutta method with the current state estimate from the Kalman filter as the initial condition.

IV. RESULTS

In this section the performance of both controllers is evaluated on an experimental testbed using 1:43 scale RC race cars. Details on the experimental setup, the particular implementation and closed-loop performance are given in the following.

A. Experimental Setup

For our experiments, we use the Kyosho *dnano* cars, which are quite sophisticated with a front and rear suspension and a rear axle differential. The cars are able to reach speeds of over 3 m/s on straights, which corresponds to an upscaled speed of about 465 km/h. The wide speed range, the high maximal forward velocity and the small scale introduce further complexity which is not present in full size testbeds.

The testbed further consists of an infrared camera tracking system, a control board and a custom built race track of length $L = 18.43$ m (measured at the center line), see Figure 7. A PointGrey Flea3 camera captures 100 frames per second with an accuracy of below 4 mm. A wide angle lens with an infrared filter is used to capture the 4 by 4 meter track. Reflecting markers are arranged on the cars in different unique patterns and used to identify the car and to measure the position and angle of the cars. An Extended Kalman Filter (EKF) is used for state estimation, filtering the vision data and estimating the longitudinal and lateral velocity as well as the yaw rate. The vision data of all cars is passed over an Ethernet connection to a control PC, which runs one of the two control algorithms described in Section III together with the corridor planner solving the DP problem. The calculated inputs are then sent via Bluetooth to the car. As the original electronics of the *dnano* cars does not feature an open digital communication link, we have replaced it by a custom made PCB, featuring a Bluetooth chip, current and voltage sensors, H-bridges

并通过使用两个平行的仿射不等式（带状区域）表示每个时间的可行位置集来处理障碍约束。这些方法的主要区别在于，第一种方法（参见第III-A节和算法Ia）是一种两级方法，具有独立的路径规划机制，并结合参考跟踪MPC控制器；而第二种方法（参见第III-B节和算法Ib）将两个任务（路径规划和路径跟踪）结合成一个基于模型预测轮廓控制的非线性最优控制问题。

Algorithm 1 HRHC and MPCC algorithm

(Ia) HRHC

- 1: get current position and velocities
- 2: compute delay compensation
- 3: **function** BORDERADJUSTMENT
- 4: shift old planned path by one stage
- 5: get position of opposing cars
- 6: run DP and get new borders (Sec. III-C)
- 7: **end function**
- 8: run path planner (4)
- 9: linearize and discretize the model, i.e. build problem (6)
- 10: solve QP using FORCES (6)
- 11: send first control at the end of time slot
- 12: **go to** step 1

(Ib) MPCC

- 1: get current position and velocities
 - 2: compute delay compensation
 - 3: augment old QP output (Section III-B4)
 - 4: **function** BORDERADJUSTMENT
 - 5: get position of opposing cars
 - 6: run DP and get new borders (Sec. III-C)
 - 7: **end function**
 - 8: linearize and discretize the model and the cost function, i.e. build problem (14)
 - 9: solve QP using FORCES (14)
 - 10: send first control at the end of time slot
 - 11: **go to** step 1
-

两个控制器在采样周期结束时发送新的控制输入。为了补偿这个固定延迟，在算法开始时将非线性模型向前仿真这一时间间隔，见算法Ia和Ib的第2行。非线性模型使用二阶Runge-Kutta方法积分，以卡尔曼滤波器的当前状态估计作为初始条件。

四、结果

在本节中，使用1:43比例的遥控赛车在实验测试平台上评估两种控制器的性能。实验设置、具体实现和闭环性能的详细信息如下所示。

A. Experimental Setup

在我们的实验中，我们使用Kyosho *dnano* 汽车，这些汽车相当精密，配有前后悬挂和后桥差速器。汽车在直道上能够达到超过3米/秒的速度，这相当于约465公里/小时的放大速度。宽广的速度范围、高最大前进速度和小比例带来了进一步的复杂性，这是全尺寸测试平台所没有的。

测试平台还包括一个红外摄像机跟踪系统、一个控制板和一个定制的赛道，赛道长度为 $L = 18.4$ 3 米（沿中心线测量），见图 7。一台 PointGrey Flea3 摄像机以每秒 100 帧的速度拍摄，精度低于 4 毫米。使用带红外滤镜的广角镜头来捕捉 4×4 米的赛道。反光标记按照不同的独特图案布置在车辆上，用于识别车辆以及测量车辆的位置和角度。采用扩展卡尔曼滤波器（EKF）进行状态估计，对视觉数据进行滤波并估计纵向和横向速度以及偏航率。所有车辆的视觉数据通过以太网连接传送到控制 PC，该控制 PC 运行第 III 节中描述的两种控制算法之一，并结合走廊规划器解决动态规划（DP）问题。计算得到的控制输入随后通过蓝牙发送给车辆。由于原车辆的电子设备无法

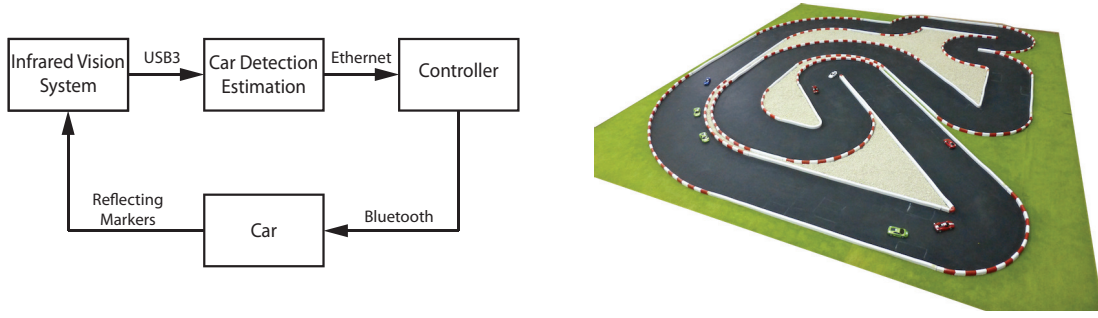


Fig. 7: Visualization of the control loop and a picture of the used track

for DC motor actuation, and an ARM Cortex M4 microcontroller for the low level control loops such as steering servo and traction motor control.

B. Implementation

The HRHC is implemented in ANSI C on an embedded platform using an ARM A9 chip (Exynos 4412) at 1.7 GHz. The chip is identical to the one used in Samsung Galaxy S3 smartphones. The runtime environment was Ubuntu Linux version 12.11, and the code was compiled with the GCC compiler 4.6.3 with option `-O2`. The sampling time of the controller is $T_s = 20$ ms, while the discretization time of the path planner and the tracking MPC controller is 25 ms. The horizon length is $N = 14$ time steps, which gives an ahead prediction of 0.35 s. The tracking MPC controller results in a multistage QP with 9 variables per stage, i.e. 132 variables in total. We use a library of 95 candidate trajectories, out of which 26 are drifting stationary velocity points.

The MPCC does not have the limitation of the HRHC path planner, or in other words, the acceleration is not zero over the whole horizon, thus it is possible to use longer horizons which should increase the performance. Hence the controller runs with a horizon length of $N = 40$ with a sampling time of $T_s = 20$ ms, which corresponds to a 0.8 s ahead prediction. With the re-linearization method presented in Section III-B4, the discretization and sampling time have to be the same, as the re-linearization is based on the last QP solution. The resulting QP has 14 variables per stage and 570 in total.

The MPCC is implemented on a newer generation embedded platform using an Exynos 5410 chip based on the ARM Cortex A15 architecture and running at 1.6 GHz. The chip is identical to the one used in the Korean version of the Samsung Galaxy S4 smartphone. The embedded board runs Ubuntu Linux version 12.04. The ANSI C code of the MPCC controller has been compiled with GCC 4.6.3 with option `-O3`. To improve the convergence of the QP, the problem data is scaled to lie between -1 and $+1$, and the objective (Hessian and linear term) is scaled by a factor of 0.1 to reduce large entries occurring due to the lag error term. Note that these scalings do not change the minimizer. The optimality criteria of the FORCES solver are adjusted such as to terminate once a sufficiently good solution has been found, with the residuals of the equality and inequality constraints set to 10^{-5} , and with the duality gap set to 10^{-4} .

C. Single Car Racing

The driven trajectories for a single car are depicted in Figure 8, with the velocity profile encoded in colors. Depicted are three laps, for which the HRHC achieves a lap time between 9.5 to 9.8 s while the MPCC yields lap times between 8.9 and 9.2 s.

To better understand the driven trajectory of the HRHC, the path planner of the HRHC has to be analyzed. The path planner has a comparably short prediction horizon, and all velocities over the whole horizon are constant, which leads to turns of constant radius. Such a path planner works fine for 90° and even up to 180° curves. However, the car tends to drive to the outer border after the curve, as this allows driving a wider radius at a faster longitudinal velocity and thus maximizes the progress. However, this is

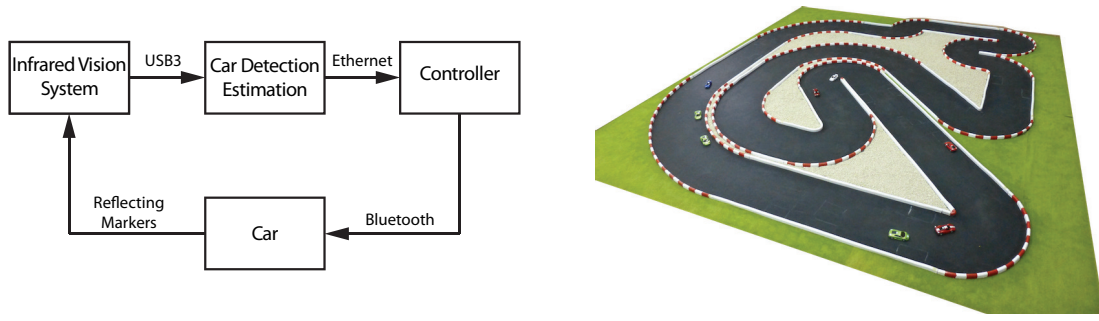


图7：控制回路的可视化及所用轨道的图片

为了直流电机驱动，以及用于低级控制回路（例如）的ARM Cortex M4微控制器尖 环形伺服和牵引电动机控制。

B. Implementation

HRHC在嵌入式平台上使用ARM A9芯片（Exynos 4412）以1.7 GHz的速度用ANSI C实现。该芯片与三星Galaxy S3智能手机中使用的芯片相同。运行环境为Ubuntu Linux 12.11版本，代码使用GCC编译器4.6.3并带有-O2选项进行编译。控制器的采样时间为 $T_s = 20$ 毫秒，而路径规划器和跟踪MPC控制器的离散化时间为25毫秒。预测时域长度为 $N = 14$ 个时间步，对应的前视预测时间为0.35秒。跟踪MPC控制器生成一个多阶段QP，每个阶段有9个变量，总计132个变量。我们使用了一个包含95条候选轨迹的库，其中26条是漂移的稳态速度点。

MPCC 不存在 HRHC 路径规划器的限制，换句话说，加速度在整个预测区间内并非为零，因此可以使用更长的预测区间，这应该会提高性能。因此，控制器以 $N = 40$ 的预测区间长度运行，采样时间为 $T_s = 20$ 毫秒，这对应于 0.8 秒的前瞻预测。使用第 III-B4 节中提出的重新线性化方法，离散化时间和采样时间必须相同，因为重新线性化是基于上一次 QP 解决方案的。最终得到的 QP 每阶段有 14 个变量，总共 570 个变量。

MPCC 在基于 Exynos 5410 芯片的新一代嵌入式平台上实现，该芯片基于 ARM Cortex A15 架构，运行频率为 1.6 GHz。该芯片与韩国版三星 Galaxy S4 智能手机使用的芯片相同。嵌入式板运行 Ubuntu Linux 12.04 版本。MPCC 控制器的 ANSI C 代码使用 GCC 4.6.3 编译，选项为 -O3。为了提高 QP 的收敛性，问题数据被缩放到介于 -1 和 $+1$ 之间，目标函数（Hessian 和线性项）按 0.1 的因子缩放，以减少由于滞后误差项产生的大数值。请注意，这些缩放不会改变最优解。FORCES 求解器的最优性标准已调整，一旦找到足够好的解即终止，等式与不等式约束的残差设置为 10^{-5} ，对偶间隙设置为 10^{-4} 。

C. Single Car Racing

单车的驾驶轨迹如图 8 所示，速度剖面用颜色编码显示。图中展示了三圈，对于这些圈，HRHC 的圈速在 9.5 到 9.8 秒之间，而 MPCC 的圈速在 8.9 到 9.2 秒之间。

为了更好地理解 HRHC 的驱动轨迹，需要分析 HRHC 的路径规划器。该路径规划器的预测范围相对较短，并且整个预测范围内的所有速度都是恒定的，这导致转弯的半径恒定。这样的路径规划器在 90° ，甚至 180° 的弯道中工作得很好。然而，车辆在弯道结束后往往会驶向外侧边界，因为这样可以以更快的纵向速度行驶更大的半径，从而最大化行驶进展。然而，这会

a limitation if the car should drive a combination of curves, where the position of the car at the end of the first curve is essential for a low overall time. This problem can be seen in Figure 8, for example in the chicane in the lower left corner. Furthermore, due to the short lookahead, the HRHC path planner is not able to prevent such an ill positioning relative to a curve ahead. Due to the suboptimal position relative to the curve, the controller has to brake more compared to a car on the ideal line and it is even possible that the car touches the borders, see the narrow 180° curve in the center of the track in Figure 9.

These limitations are not present in the MPCC approach, where a comparably long horizon is employed. Unlike the HRHC, the velocity is not fixed over the horizon but computed for each time step as the result of the NLP. As a consequence of these features, the MPCC is able to plan a path even through a complicated combination of curves. This results in a trajectory which is closer to an ideal line in a least curvature sense, especially through complicated curves like the chicane in the lower left corner of Figure 8. In this curve combination, the MPCC achieves velocities not smaller than 1 m/s, while the HRHC has to reduce the velocity to nearly 0.5 m/s to drive through the chicane, and additionally causes the car to drive an overall longer distance.

However, the MPCC has some disadvantages compared to the HRHC. In our current implementation, the MPCC still tracks the center line, which can be seen on the first straight, after the top left corner where a movement to the center line is clearly visible in Figure 8. This is clearly due to the objective function of the MPCC, and cannot be completely avoided, even if the contouring cost is small. The HRHC on the other hand does not have any cost related to the center line and only uses it as a measure of progress, thus the car drives straight in the aforementioned segment of the track. The second disadvantage can be seen in the S-curve at lower end of the track, where the MPCC has a small S shape in the driven trajectory. The HRHC on the other hand shows that the section can be driven completely straight, which gives a speed benefit. This can be most probably also attributed to the contouring error penalty, for which it would be beneficial to drive an S-shape in order to follow the center line.

In summary, our results indicate that the HRHC is limited by the path planner, and the fact that the whole horizon has the same velocity. Allowing multiple velocities within one prediction horizon would most probably increase the performance of the controller. However, the complexity in the path planner grows exponentially with the number of different velocity triples in the horizon. The MPCC, which does not have this limitation, is able to plan better trajectories which also improve the closed loop performance. However, the MPCC comes at a price, first in terms of computational cost (it is about a factor of five more expensive than the HRHC when comparing computation times on the same platform, see Section IV-E for more details on computation times) and, second, in terms of robustness. During our experiments, the re-linearization scheme turned out to be sensitive to measurement errors and model drift. In particular, sudden unexpected skidding of the car can cause problems in the re-linearization scheme. Such drifts are also responsible for the high variations between different trajectories in certain areas of the track. Furthermore, since the MPCC relies on the results from the previous iteration to compute the linearizations, sudden jumps in the high-level corridor path planner might make the previous trajectory infeasible. In such a situation, several time steps might be needed to recover feasibility of the planned trajectory, although by using the soft constrained formulation sensible inputs are still provided to the car. The HRHC on the other hand does not need any information from the last iteration as only the current position and velocity is needed. This helps to deal efficiently with unexpected behavior and fast changing measurements or corridors from the high-level obstacle avoidance mechanism.

D. Racing with multiple cars

Since both controllers plan a path around opponents while maximizing the progress, fast and safe overtaking maneuvers are enabled. In the current work, we focus on static obstacles, which seems sufficient to outperform non-expert human drivers. To robustly overtake dynamically moving obstacles (other automatic controllers or expert drivers), it would be necessary to have a prediction of their behavior, which is currently not available. These predictions could however be systematically incorporated into the

如果汽车需要驾驶一系列弯道，则存在一个限制，即汽车在第一个弯道末端的位置对于总体时间的最短化至关重要。这个问题可以在图8中看到，例如左下角的S弯。此外，由于前视距离较短，HRHC路径规划器无法避免相对于前方弯道出现这种不良位置。由于相对于弯道的位置不理想，控制器必须比行驶在理想路线上的汽车更多地刹车，甚至可能导致汽车碰到边界，参见图9中赛道中央的狭窄180°弯道。

这些限制在MPCC方法中并不存在，在该方法中使用了相对较长的预测时域。与HRHC不同，速度在整个时域内不是固定的，而是作为NLP的结果在每个时间步计算得出。由于这些特性，MPCC能够规划出即使在复杂曲线路径组合中也能行驶的路径。这导致生成的轨迹在最小曲率意义上更接近理想线路，尤其是在像图8左下角的S型弯道这样复杂的曲线中。在该曲线路径组合中，MPCC实现的速度不低于1 m/s，而HRHC则必须将速度降低到接近0.15 m/s才能通过S型弯道，并且还会导致车辆整体行驶距离更长。

然而，与HRHC相比，MPCC存在一些缺点。在我们当前的实现中，MPCC仍然会跟踪中心线，这可以在第一段直道上看到，在左上角拐弯之后，图8中可以清楚地看到车辆向中心线移动。这显然是由于MPCC的目标函数所致，即使轮廓成本很小，也无法完全避免。另一方面，HRHC没有与中心线相关的成本，仅将其用作进度的衡量，因此车辆在前述赛道段上直行。第二个缺点可以在赛道下端的S型弯道中看到，此处MPCC的行驶轨迹呈现出小S形状。相比之下，HRHC显示该路段可以完全直行，这带来了速度上的优势。这很可能也归因于轮廓误差惩罚，在这种情况下，沿中心线行驶S形状会更有利。

总之，我们的结果表明，HRHC的限制在于路径规划器，以及整个预测时域具有相同速度的事实。在一个预测时域内允许多个速度很可能会提高控制器的性能。然而，路径规划器的复杂度会随着时域中不同速度三元组数量的增加而呈指数增长。MPCC没有这个限制，能够规划出更好的轨迹，从而也改善闭环性能。然而，MPCC也有代价，首先是在计算成本方面（在同一平台上比较计算时间，MPCC大约比HRHC贵五倍，更多关于计算时间的细节见第IV-E节），其次是在鲁棒性方面。在我们的实验中，重新线性化方案对测量误差和模型漂移表现出敏感性。特别是，汽车的突然意外打滑可能会在重新线性化过程中引发问题。

D. Racing with multiple cars

由于两个控制器都计划绕过对手的路径，同时最大化进度，因此实现了快速且安全的超车操作。在当前的工作中，我们专注于静态障碍物，这似乎足以超越非专业人类驾驶员。为了稳健地超越动态移动的障碍物（其他自动控制器或专业驾驶员），有必要预测它们的行为，但目前尚不可用。然而，这些预测可以系统地纳入到

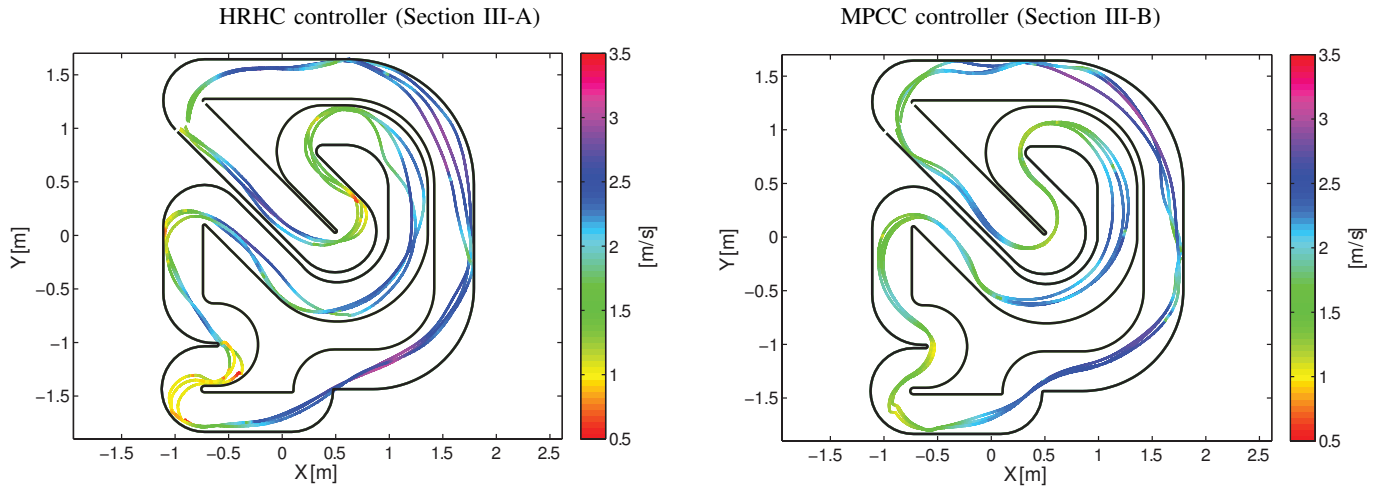


Fig. 8: The driven trajectory with velocity profile for 3 different laps. A video of the HRHC controller is available at <https://www.youtube.com/watch?v=ioKTyc9bG4c>.

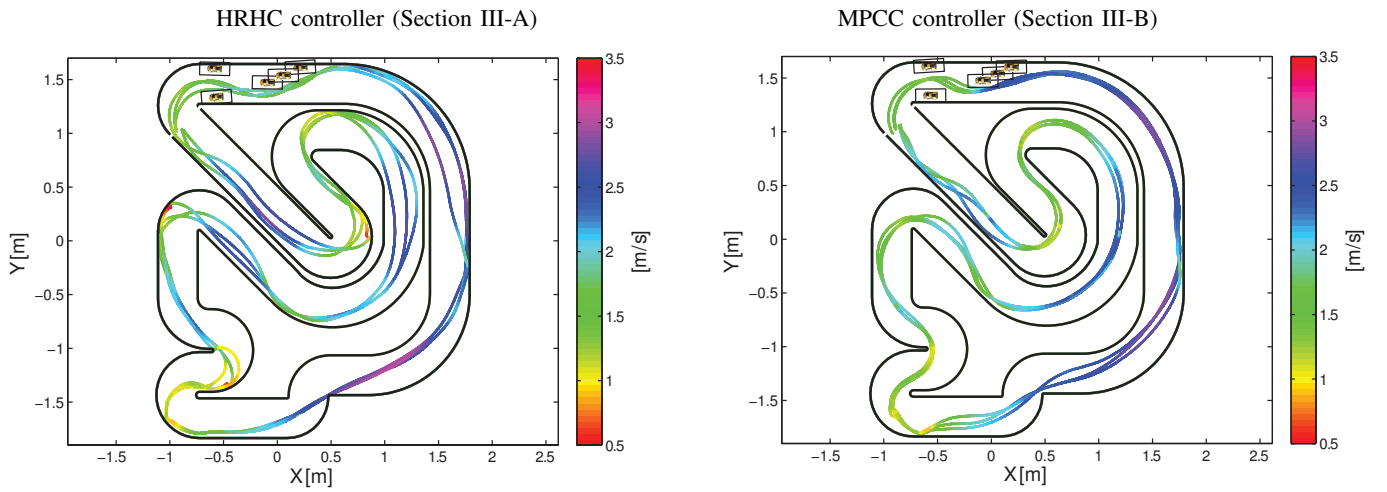


Fig. 9: Driven trajectories with static obstacles for 3 different laps. A video of the MPCC with obstacle avoidance is available at <https://www.youtube.com/watch?v=mXaEIWYQKC4>.

high-level corridor planner outlined in Section III-C, and would merely change the corridor issued to the controllers presented in Section III-A and Section III-B.

An obstacle avoidance situation is shown in Figure 9, with a detailed zoom taken at three different time points in Figure 10. These zooms show the prediction horizon and the corresponding planned trajectories. Due to the limited path planner, the HRHC is not able to directly plan a path around all obstacles from the beginning. Thus it avoids the first three cars, without predicting how to overtake the next two cars. Only after the car has made enough progress and overtaken the first group of obstacles, can the controller find a way between of the other two cars. The MPCC on the other hand plans the path around all obstacles before it even reaches the first car due to its long horizon. This is an advantage, but if the model mismatch is significant it can lead to problems, as the planned path is too optimistic.

E. Computation Times

The computation times of the most time-consuming components of the two controllers are given in Tables II and III for the laps depicted in Figure 8 (single car racing) and Figure 9 (avoidance of opponents). Note that the computation times of the individual blocks of the two controllers cannot be compared directly.

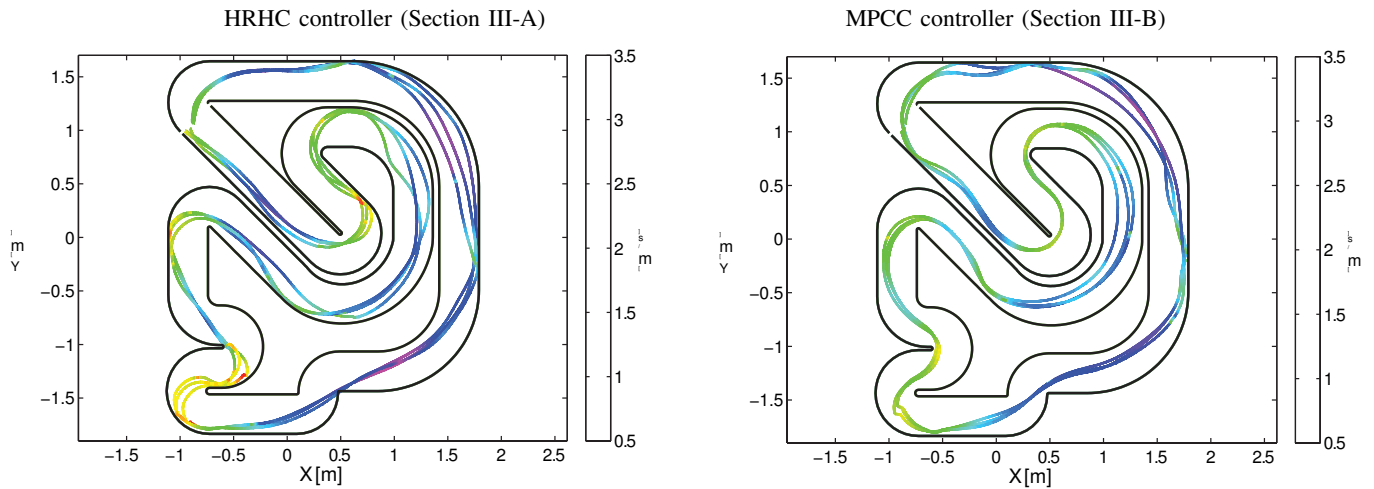


图 8: 三圈不同圈数的驱动轨迹及速度曲线。HRHC 控制器的视频可在 <https://www.youtube.com/watch?v=ioKTyc9bG4c> 观看。

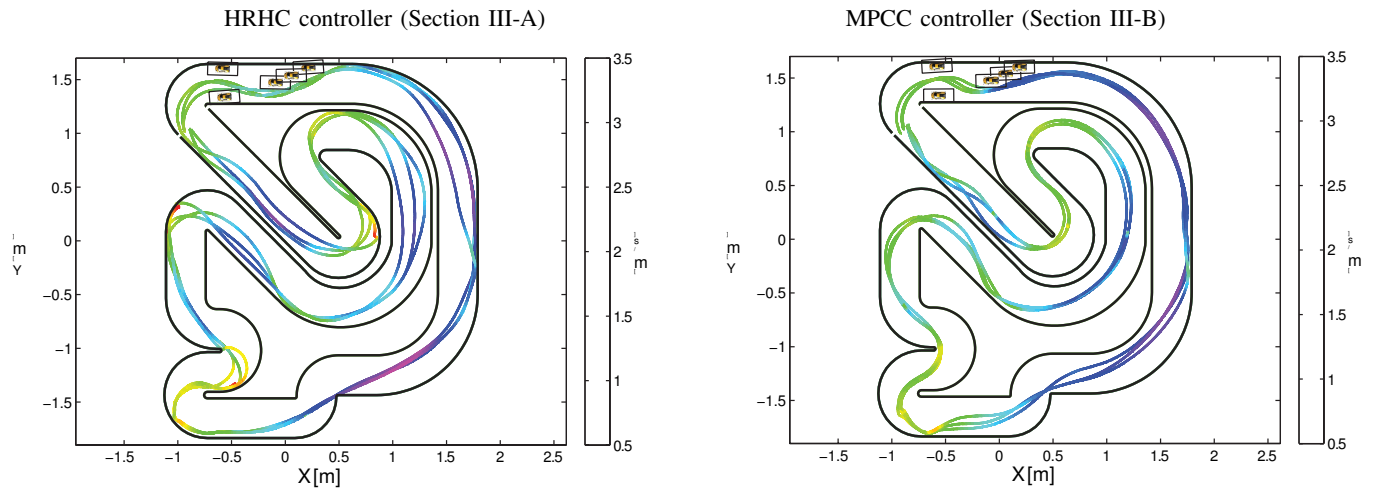


图9: 带有静态障碍物的三圈不同圈次的车辆轨迹。带有障碍物避让的MPCC视频可在 <https://www.youtube.com/watch?v=mXaElWYQKC4> 观看。

第 III-C 节中概述的高级走廊规划器，只会改变第 III-A 节和第 III-B 节中呈现给控制器的走廊。

如图9所示，展示了一个避障情况，图10则在三个不同时间点给出了详细的放大图。这些放大图显示了预测视界和相应的规划轨迹。由于路径规划器的限制，HRHC无法从一开始就直接规划绕过所有障碍物的路径。因此，它避开了前三辆车，却没有预测如何超越接下来的两辆车。只有当汽车取得足够进展并超越第一组障碍物后，控制器才能找到穿过另外两辆车之间的路径。另一方面，由于MPCC具有较长的视界，它在到达第一辆车之前就规划了绕过所有障碍物的路径。这是一个优势，但如果模型不匹配严重，可能会导致问题，因为规划的路径过于乐观。

E. Computation Times

两个控制器最耗时组件的计算时间在表 II 和表 III 中给出，分别对应图 8（单车赛车）和图 9（躲避对手）所示的圈数。请注意，两个控制器各个模块的计算时间不能直接比较。

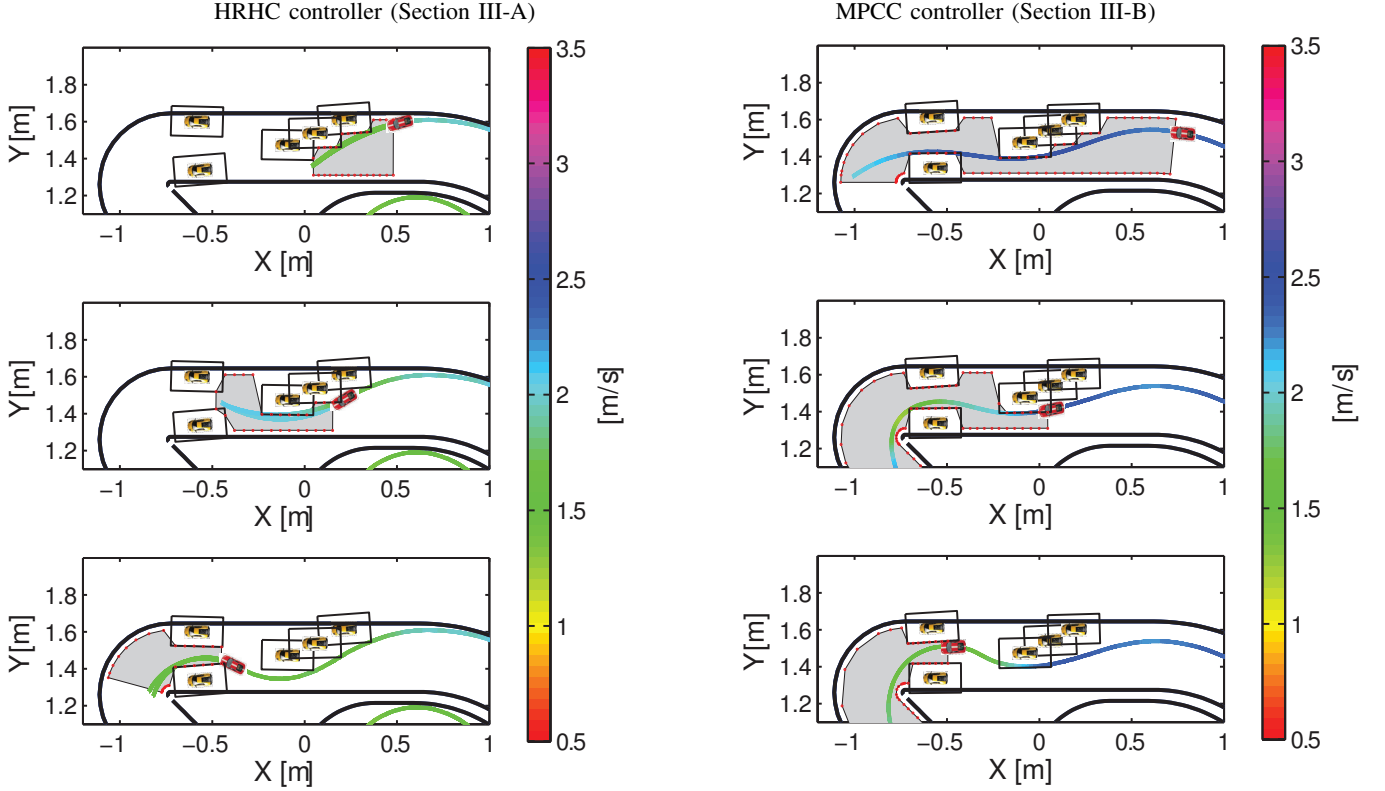


Fig. 10: Obstacle avoidance at 3 different time steps. Grey indicates the corridor chosen by the obstacle avoidance algorithm.

To directly compare the computation times of the two controllers, the horizon length should be identical. However, due to the constant velocity limitation in the HRHC path planner, a horizon length identical to the MPCC is impossible without using multiple segments of constant velocity, which would make the path planning combinatorial in complexity and thus currently prohibitive for a real-time implementation. Vice versa, the MPCC cannot work robustly with the short horizon length used by the HRHC. The multiple car case does not have a significant impact on the execution times of the lower level controller; only the path planning in the HRHC gets slightly more complicated. However, the computation time of the high-level corridor planner (the DP approach from Section III-C) varies significantly, depending on the complexity of the racing situation. For the MPCC, the limiting factor with respect to the sampling time is the computation time for solving the QP. The main bottleneck in the HRHC is the path planner, which has a very high maximal computation time caused by back up rules in the case no feasible trajectory can be found. The QP on the other hand is the most expensive step in the average, but does not have a large variation in computation time, which makes it less critical.

The 20 ms sampling time was missed by the HRHC controller in only 0.07 % (one sampling instant in three laps) and by the MPCC controller in 4.4 % (60 sampling instants in three laps) of the time. Overall, our experiments demonstrate that both schemes can be implemented in (soft) real-time at sampling rates of 50 Hz.

ACKNOWLEDGMENT

We gratefully acknowledge the work of our students in the course of the race car project: Kenneth Kuchera, Samuel Zhao and Florian Perrodin for their fruitful help in implementing the MPCC approach; Michael Janser for his contribution to the obstacle avoidance (DP) algorithm; Sandro Merkli, Nils Wenzler and Marcin Dymczyk for the development of the embedded control software and the testbed setup; Celestine Dünner and Sandro Merkli for building the track and low-level filtering software; Michael

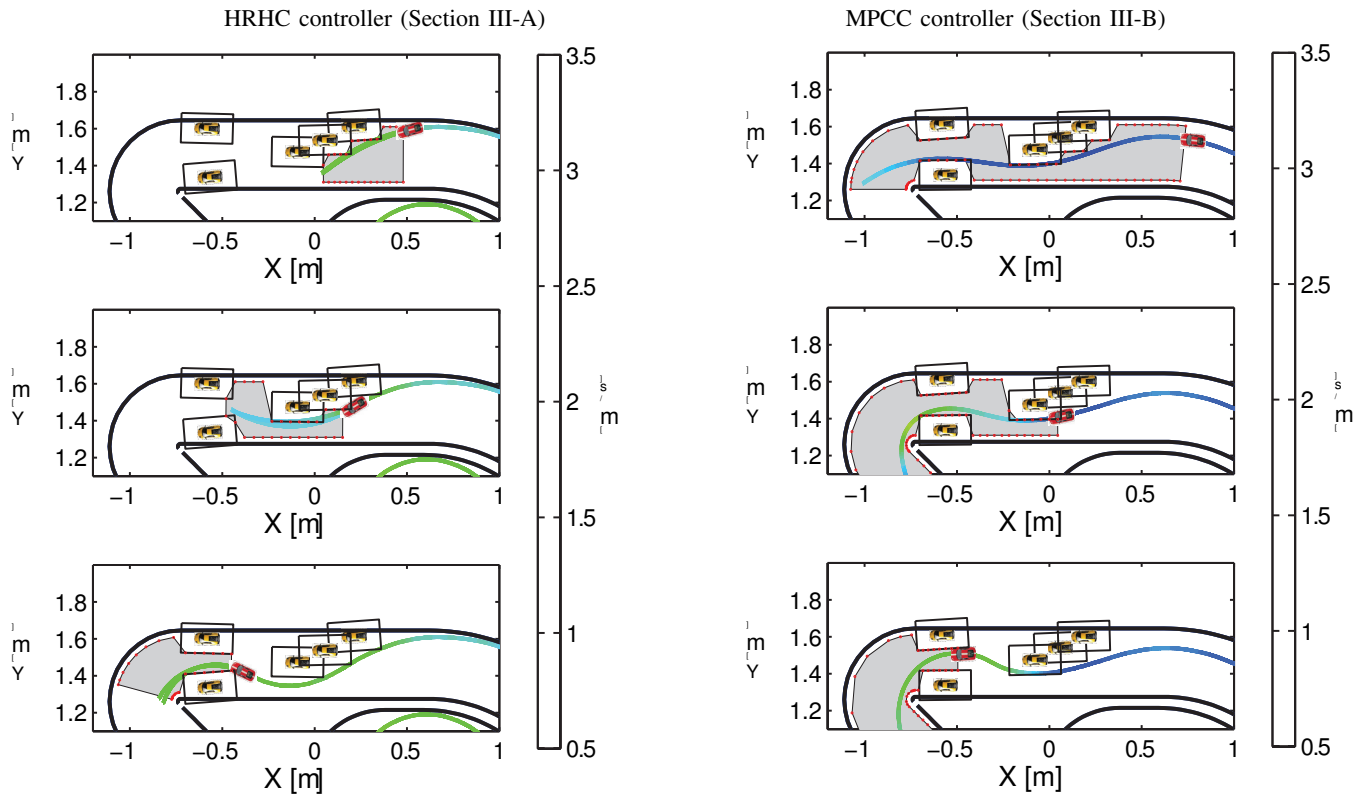


图10：在三个不同时间步的障碍物规避。灰色表示障碍物规避算法选择的路径通道。

为了直接比较这两种控制器的计算时间，视野长度应该相同。然而，由于HRHC路径规划器中恒定速度的限制，如果不使用多个恒速段，就不可能实现与MPCC相同的视野长度，而使用多个恒速段会使路径规划的复杂性呈组合式增长，因此在当前情况下不适合实时实现。反之，MPCC也无法在HRHC使用的短视野长度下稳健工作。多车情况对低级控制器的执行时间没有显著影响；只有HRHC中的路径规划会稍微复杂一些。然而，高级走廊规划器（第III-C节中的DP方法）的计算时间变化很大，这取决于比赛情况的复杂程度。对于MPCC，关于采样时间的限制因素是求解QP的计算时间。HRHC的主要瓶颈

HRHC 控制器仅在 0.07% 的时间内（在三圈中一次采样时刻）错过了 20 毫秒的采样时间，而 MPCC 控制器在 4.4% 的时间内（在三圈中 60 个采样时刻）错过了采样时间。总体而言，我们的实验表明，两种方案都可以在 50 Hz 的采样率下实现（软）实时控制。

致谢

我们衷心感谢学生们在赛车项目中的工作：Kenneth Kuchera、Samuel Zhao 和 Florian Perrodin 对 MPCC 方法实施的宝贵帮助；Michael Janser 对障碍物规避（DP）算法的贡献；Sandro Merkli、Nils Wenzler 和 Marcin Dymczyk 对嵌入式控制软件开发和测试平台搭建的贡献；Celestine Dünner 和 Sandro Merkli 对赛道建设及低级过滤软件的建设；Michael

TABLE II: HRHC computation times in milliseconds

Without Obstacles	Mean	Stdev	Max	With Obstacles	Mean	Stdev	Max
Border Adjustment	0.02	0.01	0.04	Border Adjustment	1.33	0.37	2.45
Path Planning	2.93	1.96	13.47	Path Planning	3.18	2.04	9.76
QP Generation	0.52	0.09	1.32	QP Generation	0.53	0.06	0.76
QP with FORCES	5.10	0.73	7.56	QP with FORCES	5.11	0.92	9.25

TABLE III: MPCC computation times in milliseconds

Without Obstacles	Mean	Stdev	Max	With Obstacles	Mean	Stdev	Max
Border Adjustment	0.34	0.11	0.64	Border Adjustment	0.70	0.24	1.05
QP Generation	1.95	0.44	2.92	QP Generation	2.34	0.37	2.79
QP with FORCES	15.03	1.61	21.16	QP with FORCES	14.43	1.40	21.46

Dahinden and Christian Stocker for the initial design of the embedded PCB board; Benjamin Keiser for the current version of the infrared vision system. We furthermore thank Sean Summers, Nikolaos Kariotoglou, Christian Conte and John Lygeros from the Automatic Control Laboratory for supervising various projects and providing important comments to the methods presented in this paper. Special thanks go to Colin Jones, who originated the idea of the race car testbed.

The research leading to these results has received funding from the EU in FP7 via EMBOCON (ICT-248940).

REFERENCES

- [1] T. Faulwasser, B. Kern, and R. Findeisen, "Model predictive path-following for constrained nonlinear systems," in *Proceedings of the 48th IEEE Conference on Decision and Control, 2009 held jointly with the 2009 28th Chinese Control Conference*, pp. 8642–8647, 2009.
- [2] D. Lam, C. Manzie, and M. Good, "Model predictive contouring control," in *Conference on Decision and Control (CDC)*, pp. 6137–6142, 2010.
- [3] A. Domahidi, A. Zraggen, M. Zeilinger, M. Morari, and C. Jones, "Efficient interior point methods for multistage problems arising in receding horizon control," in *Conference on Decision and Control (CDC)*, (Maui, HI, USA), pp. 668–674, Dec. 2012.
- [4] A. Domahidi, "FORCES: Fast optimization for real-time control on embedded systems." <http://forces.ethz.ch>, Oct. 2012.
- [5] R. Horowitz and P. Varaiya, "Control design of an automated highway system," *Proceedings of the IEEE*, vol. 88, no. 7, pp. 913–925, 2000.
- [6] M. Buehler, K. Iagnemma, and S. Singh, *The 2005 DARPA grand challenge: the great robot race*, vol. 36. Springer, 2007.
- [7] M. Buehler, K. Iagnemma, and S. Singh, *The DARPA urban challenge: Autonomous vehicles in city traffic*, vol. 56. Springer, 2009.
- [8] K. Kritayakirana and C. Gerdes, "Using the centre of percussion to design a steering controller for an autonomous race car," *Vehicle System Dynamics*, vol. 15, pp. 33–51, 2012.
- [9] E. Velenis, E. Frazzoli, and P. Tsiotras, "On steady-state cornering equilibria for wheeled vehicles with drift," in *Proceedings of the 48th IEEE Conference on Decision and Control, 2009 held jointly with the 2009 28th Chinese Control Conference*, pp. 3545–3550, IEEE, 2009.
- [10] C. Voser, R. Y. Hindiyeh, and J. C. Gerdes, "Analysis and control of high sideslip manoeuvres," *Vehicle System Dynamics*, vol. 48, no. S1, pp. 317–336, 2010.
- [11] F. Borrelli, P. Falcone, T. Keviczky, and J. Asgari, "MPC-based approach to active steering for autonomous vehicle systems," *International Journal of Vehicle Autonomous Systems*, vol. 3, no. 2, pp. 265–291, 2005.
- [12] P. Falcone, F. Borrelli, J. Asgari, H. E. Tseng, and D. Hrovat, "Predictive active steering control for autonomous vehicle systems," *IEEE Transactions on Control Systems Technology*, vol. 15, no. 3, pp. 566–580, 2007.
- [13] T. Besselmann and M. Morari, "Hybrid parameter-varying model predictive control for autonomous vehicle steering," *European Journal of Control*, vol. 14, no. 5, pp. 418–431, 2008.
- [14] J. Ackermann, J. Guldner, W. Sienel, R. Steinhauser, and V. I. Utkin, "Linear and nonlinear controller design for robust automatic steering," *IEEE Transactions on Control Systems Technology*, vol. 3, no. 1, pp. 132–143, 1995.
- [15] S. Fuchshumer, K. Schlacher, and T. Rittenschober, "Nonlinear vehicle dynamics control - a flatness based approach," in *Conference on Decision and Control (CDC)*, pp. 6492–6497, 2005.
- [16] Y. Gao, T. Lin, F. Borrelli, E. Tseng, and D. Hrovat, "Predictive control of autonomous ground vehicles with obstacle avoidance on slippery roads," *Proceedings of DSCC*, 2010.

表 II: HRHC 计算时间 (毫秒)

Without Obstacles	Mean	Stdev	Max	With Obstacles	Mean	Stdev	Max
Border Adjustment	0.02	0.01	0.04	Border Adjustment	1.33	0.37	2.45
Path Planning	2.93	1.96	13.47	Path Planning	3.18	2.04	9.76
QP Generation	0.52	0.09	1.32	QP Generation	0.53	0.06	0.76
QP with FORCES	5.10	0.73	7.56	QP with FORCES	5.11	0.92	9.25

表 III: MPCC 计算时间 (毫秒)

Without Obstacles	Mean	Stdev	Max	With Obstacles	Mean	Stdev	Max
Border Adjustment	0.34	0.11	0.64	Border Adjustment	0.70	0.24	1.05
QP Generation	1.95	0.44	2.92	QP Generation	2.34	0.37	2.79
QP with FORCES	15.03	1.61	21.16	QP with FORCES	14.43	1.40	21.46

感谢Dahinden和Christian Stocker对嵌入式PCB板的最初设计；感谢Benjamin Keiser对当前版本红外视觉系统的贡献。我们还感谢自动控制实验室的Sean Summers、Nikolaos Kariotoglou、Christian Conte和John Lygeros，他们监督了各类项目并对本文提出的方法提供了重要的建议。特别感谢Colin Jones，他提出了赛车测试平台的创意。

T导致这些结果的研究已通过 EMBOCON (ICT-) 在欧盟第七框架计划 (FP7) 获得资助 2044018

参考文献

- [1] T. Faulwasser, B. Kern, 和 R. Findeisen, “约束非线性系统的模型预测路径跟随”，发表于 *Proceedings of the 48th IEEE Conference on Decision and Control, 2009 held jointly with the 2009 28th Chinese Control Conference*, 第 8642–8647 页, 2009 年。[2] D. Lam, C. Manzie, 和 M. Good, “模型预测轮廓控制”，发表于 *Conference on Decision and Control (CDC)*, 第 6137–6142 页, 2010 年。[3] A. Domahidi, A. Zraggen, M. Zeilinger, M. Morari, 和 C. Jones, “递推视野控制中多阶段问题的高效内点法”，发表于 *Conference on Decision and Control (CDC)*, (美国夏威夷茂宜岛), 第 668–674 页, 2012 年 12 月。[4] A. Domahidi, “FORCES: 嵌入式系统实时控制的快速优化。” <http://forces.ethz.ch>, 2012 年 10 月。[5] R. Horowitz 和 P. Varaiya, “自动化高速公路系统的控制设计”，*Proceedings of the IEEE*, 第 88 卷, 第 7 期, 第 913–925 页, 2000 年。[6] M. Buehler, K. Iagnemma, 和 S. Singh, *The 2005 DARPA grand challenge: the great robot race*, 第 36 卷。施普林格, 2007 年。[7] M. Buehler, K. Iagnemma, 和 S. Singh, *The DARPA urban challenge: Autonomous vehicles in city traffic*, 第 56 卷。施普林格, 2009 年。[8] K. Kritayakirana 和 C. Gerdes, “利用击打中心设计自动赛车的转向控制器”，*Vehicle System Dynamics*

- [17] A. Gray, Y. Gao, T. Lin, J. K. Hedrick, H. E. Tseng, and F. Borrelli, "Predictive control for agile semi-autonomous ground vehicles using motion primitives," in *American Control Conference (ACC)*, pp. 4239–4244, IEEE, 2012.
- [18] E. Frazzoli, M. Dahleh, and E. Feron, "Maneuver-based motion planning for nonlinear systems with symmetries," *IEEE Transactions on Robotics*, vol. 21, no. 6, pp. 1077–1091, 2005.
- [19] E. Velenis, P. Tsiotras, and J. Lu, "Modeling aggressive maneuvers on loose surfaces: The cases of trail-braking and pendulum-turn," in *European Control Conference*, pp. 1233–1240, 2007.
- [20] J. Frasch, A. Gray, M. Zanon, H. Ferreau, S. Sager, F. Borrelli, and M. Diehl, "An auto-generated nonlinear MPC algorithm for real-time obstacle avoidance of ground vehicles," in *Proceedings of the European Control Conference*, no. accepted, 2013.
- [21] M. Diehl, H. Bock, J. Schlöder, R. Findeisen, Z. Nagy, and F. Allgöwer, "Real-time optimization and nonlinear model predictive control of processes governed by differential-algebraic equations," *Journal of Process Control*, vol. 12, no. 4, pp. 577–585, 2002.
- [22] J. Jeon, S. Karaman, and E. Frazzoli, "Anytime computation of time-optimal off-road vehicle maneuvers using the RRT*," in *Conference on Decision and Control (CDC)*, pp. 3276–3282, 2011.
- [23] J. Jeon, R. Cowlagi, S. Peters, S. Karaman, E. Frazzoli, P. Tsiotras, and K. Iagnemma, "Optimal motion planning with the half-car dynamical model for autonomous high-speed driving," in *American Control Conference (ACC)*, pp. 188–193, 2013.
- [24] E. Bakker, L. Nyborg, and H. Pacejka, "Tyre modelling for use in vehicle dynamics studies," *SAE*, 1987.
- [25] D. G. Luenberger, *Linear and nonlinear programming*. Springer, 2003.
- [26] E. C. Kerrigan and J. M. Maciejowski, "Soft constraints and exact penalty functions in model predictive control," in *Proc. UKACC International Conference (Control 2000)*, (Cambridge, UK), September 2000.
- [27] Y. Koren, "Control of machine tools," *Transactions of the ASME Journal of Manufacturing Science and Engineering*, vol. 119, 1997.
- [28] R. C. Ko, M. C. Good, and S. K. Holgamuge, "Adaptive calibration of feedforward controllers for laser profiling machines," in *Proceedings of Information, Decision and Control*, 1999.
- [29] M. Diehl, H. G. Bock, and J. P. Schlöder, "A real-time iteration scheme for nonlinear optimization in optimal feedback control," *SIAM Journal on Control and Optimization*, vol. 43, no. 5, pp. 1714–1736, 2005.
- [30] D. P. Bertsekas, *Dynamic programming and optimal control*, vol. 1. Athena Scientific Belmont, 1995.
- [31] D. Ferguson, T. M. Howard, and M. Likhachev, "Motion planning in urban environments," *Journal of Field Robotics*, vol. 25, pp. 939–960, 2008.
- [32] M. Montemerlo, J. Becker, S. Bhat, H. Dahlkamp, D. Dolgov, S. Ettinger, D. Haehnel, T. Hilden, G. Hoffmann, B. Huhnke, and et al., "Junior: The stanford entry in the urban challenge," *Journal of Field Robotics*, vol. 25, pp. 569–597, 2008.
- [33] Y. Kuwata, G. A. Fiore, J. Teo, E. Frazzoli, and J. P. How, "Motion planning for urban driving using RRT," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 1681–1686, 2008.

[17] A. Gray, Y. Gao, T. Lin, J. K. Hedrick, H. E. Tseng, 和 F. Borrelli, “使用运动基元对敏捷半自主地面车辆的预测控制,” 发表于 *American Control Conference (ACC)*, 页 4239–4244, IEEE, 2012年。 [18] E. Frazzoli, M. Dahleh, 和 E. Feron, “具有对称性的非线性系统基于机动的运动规划,” *IEEE Transactions on Robotics*, 卷 21, 期 6, 页 1077–1091, 2005年。 [19] E. Velenis, P. Tsiotras, 和 J. Lu, “在松散表面上建模激进机动: 拖尾制动和摆动转弯的情况,” 发表于 *European Control Conference*, 页 1233–1240, 2007年。 [20] J. Frasch, A. Gray, M. Zanon, H. Ferreau, S. Sager, F. Borrelli, 和 M. Diehl, “一种用于地面车辆实时避障的自动生成非线性MPC算法,” 发表于 *Proceedings of the European Control Conference*, 号已接受, 2013年。 [21] M. Diehl, H. Bock, J. Schlöder, R. Findeisen, Z. Nagy, 和 F. Allgöwer, “受微分-代数方程控制的过程的实时优化和非线性模型预测控制,” *Journal of Process Control*, 卷 12, 期 4, 页 577–585, 2002年。 [22] J. Jeon